Tab 1

Train-Test Split in Machine Learning

What is a Train-Test Split?

Train-Test Split is a data preprocessing technique used in Machine Learning, where the dataset is divided into two parts:

- **Training Set** → Used to train (teach) the machine learning model.
- **Testing Set** → Used to evaluate how well the trained model performs on new, unseen data.

The main purpose of a train-test split is to check whether the model has **learned the underlying pattern** or has simply **memorised the training data (overfitting)**.

Why Do We Need Train-Test Split?

Imagine a student preparing for an examination.

- The student studies from a textbook.
- During the exam, the questions are different but based on the same concepts.

If the exam contains the same questions that the student memorised, getting high marks does not prove the student understands the subject.

Machine Learning works in the same way.

- **Training Data = Textbook**
- **Testing Data = Examination**

The model should perform well on **new data**, not only on the data it has already seen.

Example Dataset

Suppose we have a dataset for

predicting house prices House Size (sq.ft.)	Bedrooms	Price (Lakhs)
800	2	35
950	2	40
1200	3	55
1400	3	65
1600	3	72
1800	4	85
2000	4	95
2200	5	110
2500	5	125
2800	5	140

There are **10 records**.

Splitting the Dataset

Suppose we choose an **80:20 split**.

Training Set (80%)

House Size	Bedrooms	Price
800	2	35
950	2	40
1200	3	55
1400	3	65
1600	3	72
1800	4	85
2000	4	95
2200	5	110

Total = 8 records

The model learns the relationship between:

- House Size
- Bedrooms
- House Price

Testing Set (20%)

House Size	Bedrooms	Price
2500	5	125
2800	5	140

Total = 2 records

These records are **hidden from the model during training**.

After training, the model predicts their prices.

Example:

Actual Price	Predicted Price
125	123
140	142

Since the predictions are close to the actual prices, we conclude that the model generalises well to unseen data.

Common Train-Test Ratios

Training	Testing	When Used
70%	30%	Small datasets
75%	25%	General purpose
80%	20%	Most commonly used
90%	10%	Very large datasets

There is no single best ratio; the choice depends on the size of the dataset and the problem being solved.

What Happens Without Train-Test Split?

Suppose we train and test using the **same dataset**.

Training Accuracy = **100%**

This looks excellent.

Now the model is given completely new data.

Accuracy drops to **65%**.

Why?

Because the model **memorised** the training examples instead of learning the actual pattern. This problem is called **Overfitting**. Using a separate test set helps detect this issue.

Python Example (Scikit-Learn)

```
from sklearn.model_selection import train_test_split

# Features

X = data.drop("Price", axis=1)

# Target

y = data["Price"]

# Split the dataset

X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.2,random_state=42)
```

Explanation

- `X` → Input features
- `y` → Target/output
- `test_size=0.2` → 20% of the data is used for testing
- `random_state=42` → Ensures the same random split every time the code is run for reproducible results.

Advantages of Train-Test Split

- Evaluates model performance on unseen data.
- Detects overfitting.
- Simulates real-world predictions.
- Simple and computationally efficient.
- Widely used as the first step in supervised machine learning workflows.

Tab 2

Machine Learning

What is Machine Learning?

Machine Learning (ML) is a branch of **Artificial Intelligence (AI)** that enables computers to learn from data and improve their performance without being explicitly programmed.

Instead of writing rules manually, we provide data to the machine, and it learns patterns from the data to make predictions or decisions.

Definition

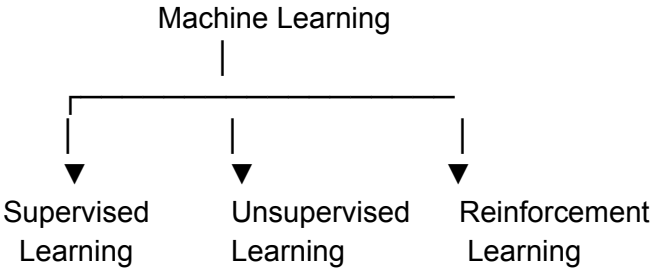
> Machine Learning is the field of computer science that allows computers to learn from past data and make predictions or decisions without being explicitly programmed.

Real-Life Applications of Machine Learning

- Email Spam Detection
- Netflix & YouTube Recommendations
- Face Recognition
- Self-Driving Cars
- Medical Disease Prediction
- Weather Forecasting
- Online Shopping Recommendations
- Fraud Detection in Banking
- Voice Assistants (Alexa, Siri, Google Assistant)

Types of Machine Learning

Machine Learning is mainly divided into **three categories**:



1. Supervised Learning

Definition

Supervised Learning is a type of machine learning where the model is trained using **labelled data**.

A labelled dataset contains both:

- Input (Features)
- Correct Output (Target)

The model learns the relationship between input and output and predicts the output for new data.

Structure

Input (X) ----------> Machine Learning Model ----------> Output (Y)
(Training with Correct Answers)

Example

Hours Studied	Exam Marks
2	40
4	60

6	75
8	90

If a student studies **5 hours**, the model predicts approximately **68 marks**.

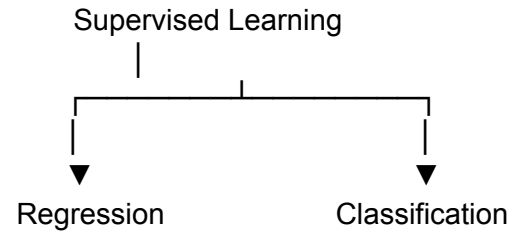
Since the correct output (marks) is already available during training, this is **Supervised Learning**.

Applications

- House Price Prediction
- Student Marks Prediction
- Salary Prediction
- Email Spam Detection
- Disease Prediction
- Weather Forecasting

Types of Supervised Learning

Supervised Learning is divided into **two categories**:



A. Regression

Regression predicts **continuous numerical values**.

Output

10.5
₹25,00,000

85.6
6.8 LPA

Examples

- House Price Prediction
- Student Salary Prediction
- Temperature Prediction
- Sales Forecasting

B. Classification

Classification predicts **categories or classes**.

Output

Yes / No

Spam / Not Spam

Male / Female

Positive / Negative

Cat / Dog

Examples

- Email Spam Detection
- Disease Diagnosis
- Face Recognition
- Sentiment Analysis

Regression vs Classification

Regression	Classification
Predicts numerical values	Predicts categories

Output is continuous	Output is discrete
Example: House Price	Example: Spam Detection
Example: Salary	Example: Disease Prediction

2. Unsupervised Learning

Definition

Unsupervised Learning is a type of machine learning where the model is trained using **unlabeled data**.

The dataset contains only input features.

There are **no correct answers** provided.

The model automatically discovers hidden patterns or groups within the data.

Structure

Input Data



Machine Learning Algorithm



Hidden Patterns / Groups

Example

Suppose a shopping mall has customer information.

Customer	Age	Annual Income
A	22	₹2 Lakhs

B	25	₹2.5 Lakhs
C	50	₹10 Lakhs
D	48	₹9.5 Lakhs

The algorithm automatically groups customers into:

- Young Customers
- High-Income Customers

No labels are provided.

Applications

- Customer Segmentation
- Market Basket Analysis
- Recommendation Systems
- Fraud Detection
- Image Compression

Common Algorithms

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN
- PCA (Principal Component Analysis)

3. Reinforcement Learning

Definition

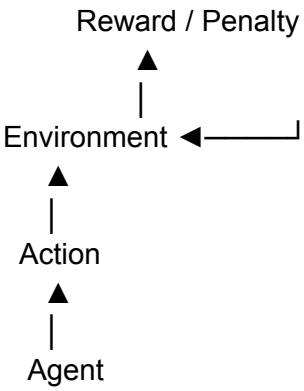
Reinforcement Learning is a type of machine learning in which an **agent learns by interacting with an environment**.

The agent performs actions and receives:

- Reward (Correct Action)
- Penalty (Wrong Action)

The objective is to maximise the total reward over time.

Structure



Example

Teaching a dog tricks.

- Correct action → Give a treat (Reward)
- Wrong action → No treat (Penalty)

The dog gradually learns the correct behaviour.

Similarly, reinforcement learning agents learn the best actions through rewards and penalties.

Applications

- Self-Driving Cars
- Robotics
- Game Playing (Chess, Go)
- Traffic Signal Optimisation
- Drone Navigation
- Resource Management

Comparison of Machine Learning Types

Feature	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Training Data	Labeled	Unlabeled	Environment Interaction
Output	Prediction	Groups/Patterns	Best Action
Learns From	Input + Correct Output	Input Only	Rewards and Penalties
Goal	Predict Output	Find Hidden Patterns	Maximize Reward
Example	House Price Prediction	Customer Segmentation	Self-Driving Cars

Which Type Should We Use?

Problem	Machine Learning Type
Predict House Price	Regression (Supervised)
Predict Student Salary	Regression (Supervised)
Spam Email Detection	Classification (Supervised)
Disease Detection	Classification (Supervised)
Customer Segmentation	Unsupervised
Product Recommendation	Unsupervised
Chess Playing AI	Reinforcement Learning
Robot Navigation	Reinforcement Learning

Key Points to Remember

- **Machine Learning** enables computers to learn from data without explicit programming.
- There are **three main types** of Machine Learning:
    - **Supervised Learning**
    - **Unsupervised Learning**
    - **Reinforcement Learning**
- **Supervised Learning** uses **labeled data** and is divided into:
    - **Regression** (predicts numerical values)
    - **Classification** (predicts categories)
- **Unsupervised Learning** works with **unlabeled data** to discover hidden patterns or groups.
- **Reinforcement Learning** learns by interacting with an environment using **rewards and penalties**.

Tab 3

Regression Analysis in Machine Learning

What is Regression?

Regression is a **Supervised Machine Learning** technique used to **predict continuous (numerical) values** based on one or more input features.

Unlike classification, which predicts categories (Yes/No, Spam/Not Spam), regression predicts a **numeric value**.

Definition

> Regression is a machine learning algorithm that finds the relationship between the independent variable(s) (input features) and the dependent variable (target/output) to predict continuous values.

Real-Life Examples of Regression

Input (Feature)	Output (Target)
House Size	House Price
Student CGPA	Salary Package
Years of Experience	Salary
Temperature	Electricity Consumption
Advertising Cost	Product Sales
Age of Car	Selling Price

In all these examples, the output is a **number**, making them regression problems.

Regression vs Classification

Regression	Classification
Predicts numerical values	Predicts categories or classes
Output is continuous	Output is discrete
Example: House Price = ₹25 Lakhs	Example: Email = Spam or Not Spam
Example: Salary = ₹8.5 Lakhs	Example: Disease = Positive or Negative

Types of Regression Algorithms

There are many regression algorithms available in Machine Learning.

1. Simple Linear Regression

Uses **one independent variable** to predict one dependent variable.

Example:

Predict Salary using Years of Experience.

Experience -----> Salary

Equation

$Y=mX+cY = mX + cY=mX+c$

Where

- **Y** = Predicted Output
- **X** = Input Feature
- **m** = Slope
- **c** = Intercept

2. Multiple Linear Regression

Uses **more than one independent variable**.

Example

Predict House Price using:

- Area
- Number of Bedrooms
- Parking Space
- Location

Area
Bedrooms
Parking
Location
↓
House Price

3. Polynomial Regression

Used when the relationship between variables is **curved** rather than linear.

Example

Predict Crop Yield based on Rainfall.

The graph forms a curve instead of a straight line.

4. Decision Tree Regression

Uses a decision tree to predict continuous values.

Example

Predict Car Price using:

- Brand
- Fuel Type
- Age
- Kilometers Driven

5. Random Forest Regression

Uses **multiple decision trees**.

The final prediction is the **average of all trees**.

Advantages

- Higher accuracy
- Less overfitting
- Works well with complex datasets

6. Support Vector Regression (SVR)

Uses the concept of Support Vector Machines for regression problems.

Suitable for:

- Small datasets
- Non-linear relationships

7. K-Nearest Neighbour Regression (KNN Regression)

Predicts the output by looking at the **K nearest data points**.

Example

If nearby houses are priced around ₹40 Lakhs, a new nearby house will likely have a similar price.

8. Ridge Regression

Used when the dataset contains **many correlated features**.

It reduces model complexity and helps prevent **overfitting**.

9. Lasso Regression

Similar to Ridge Regression but can automatically remove less important features by making some coefficients exactly zero.

Useful for **feature selection**.

Simple Linear Regression

Definition

Simple Linear Regression is a regression algorithm that uses **one independent variable** to predict **one dependent variable**.

Example

Predict a student's salary package based on CGPA.

CGPA	Salary Package (LPA)
6.5	3.0
7.0	4.2
7.5	5.0
8.0	6.1
8.5	7.3
9.0	8.4

Here,

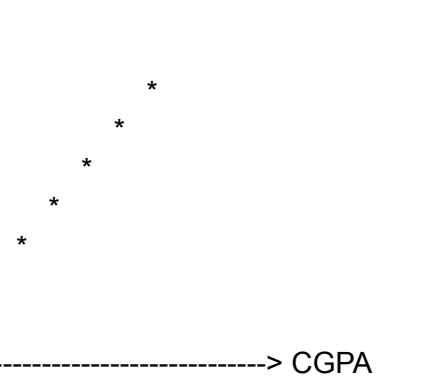
Independent Variable (X)

CGPA

Dependent Variable (Y)

Salary Package

Graph Representation



The regression algorithm tries to draw the **best-fit straight line** through the data points.

Regression Line

The straight line is called the **Regression Line** or **Best Fit Line**.

Its equation is:

$Y=mX+cY = mX + cY=mX+c$

Where

- **Y** = Predicted value
- **X** = Input value
- **m** = Slope of the line
- **c** = Y-intercept

Understanding the Equation

Suppose

$Y=2X+1Y = 2X + 1Y=2X+1$

If

$X = 4$

Then

$Y = 2 \times 4 + 1$
$Y = 9$

So, when the input is **4**, the predicted output is **9**.

Example: Student Placement Prediction

Dataset

CGPA	Package (LPA)
6.8	3.2
7.2	4.0
7.8	5.3
8.2	6.5
8.8	7.9

Suppose a student has:

CGPA = 8.0

The regression model predicts:

Package ≈ 5.9 LPA

This is a **continuous numerical prediction**, so it is a regression problem.

First, we import the foundational ecosystem for data handling and scientific tracking.

Python
```
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Read the file via Raw String (r) to handle path backslashes cleanly
df = pd.read_csv(r"D:\datasets\placement.csv")

# Quick inspect
print(df.head(3))
```

	CGPA	Package (LPA)
0	6.89	3.26
1	8.12	4.21
2	5.97	2.24

Step 2: Exploratory Data Analysis (EDA) & The Linearity Check

Before throwing data into a linear model, you must validate the underlying mathematical assumption: **Linearity**. If the relationship isn't linear, a linear model will yield an *underfitted* layout.

Python

```
# Check dimensionality and missing records
print(df.shape)          # Output: (200, 2)
print(df.isnull().sum()) # Output: CGPA: 0, Package: 0

# Linearity Check via Scatter Plot
plt.figure(figsize=(5, 4))
sns.scatterplot(x='CGPA', y='Package', data=df)
plt.title("CGPA vs Placement Package")
plt.show()
```

> **Takeaway:** The scatter plot displays a clear **upward, positive linear trajectory**. As the independent variable (CGPA) increases, the dependent variable (Package) steps up proportionally. This confirms that the dataset meets the prerequisites for linear modelling.

Step 3: Feature-Target Extraction (X/y) & Dimensionality Alignment

Scikit-Learn estimators structurally mandate that your feature matrix X must be **Multi-Dimensional** (a 2D Array / DataFrame), whereas the target vector y must remain **1-Dimensional** (a 1D Series / Array).

Python
```
# Extracting X as a 2D matrix using double brackets
X = df[['CGPA']]
print(X.ndim)  # Output: 2

# Extracting y as a 1D vector
y = df['Package']
print(y.ndim)  # Output: 1
```

Step 4: The Evaluation Partition (Train-Test Split)

To evaluate the model's actual predictive capability on unobserved data, the data matrix is split. Here, we allocate 80% for pattern discovery and 20% for the final testing evaluation.

Python
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 5: Model Initalization & Training

When we invoke .fit(), the engine optimizes the cost function (Mean Squared Error) using the training slices. It derives the absolute best fit values for our slope (m) and intercept (c).

Python
```
from sklearn.linear_model import LinearRegression

# Instantiate estimator object
lr = LinearRegression()

# Training Phase
lr.fit(X_train, y_train)
```

3. Extracting the Mathematical Equation

Every trained linear regression model can be written out as a basic algebra function:

$$y = mx + c$$
We extract the derived slope (m) and intercept (c) parameters directly from Scikit-Learn properties:

Python
```
m = lr.coef_[0]
c = lr.intercept_

print(f"Slope (m): {m}")        # Derived coefficient
print(f"Intercept (c): {c}")    # Derived intercept
```

Verifying the .predict() Engine Behind the Scenes

When you call lr.predict([[6.89]]), the codebase doesn't perform magic; it runs the simple scalar formula above.

Python
```
# Scenario A: Using Scikit-Learn's predictor engine
api_pred = lr.predict([[6.89]])

# Scenario B: Manual mathematical substitution
manual_pred = (m * 6.89) + c

print(api_pred == manual_pred) # Evaluates to True/Identical
```

4. Visualising the Best Fit Line

To physically view the optimised regression boundary against our data points, we map the model's overall prediction line across our scatter grid.

Python
```
plt.figure(figsize=(6, 4))

# Plot the real data coordinates
sns.scatterplot(x='CGPA', y='Package', data=df, color='blue', label='Original Data')

# Overlay the optimised regression path across all coordinates
plt.plot(X_train, lr.predict(X_train), color='red', linewidth=2, label='Regression Line (Best Fit)')

plt.xlabel("CGPA")
plt.ylabel("Package (LPA)")
plt.legend()
plt.show()
```

5. Model Evaluation Metrics

To judge the accuracy of our optimised line, we extract its performance metric on unseen data:

Python
accuracy_score = lr.score(X_test, y_test)
print(f"Model R-Squared Score: {accuracy_score * 100:.2%}")

Multiple Linear Regression (MLR)

Introduction

In **Simple Linear Regression**, we use **only one independent variable** to predict the dependent variable.

However, in many real-world problems, a single feature is not enough to make an accurate prediction. Multiple factors influence the output.

To solve this problem, we use **Multiple Linear Regression (MLR)**.

Definition

Multiple Linear Regression (MLR) is a supervised machine learning algorithm that predicts a **continuous numerical value** using **two or more independent variables (features)**.

In Simple Words

> Multiple Linear Regression predicts one output using multiple input variables.

Formula of Multiple Linear Regression

The mathematical equation of Multiple Linear Regression is:

$Y=\beta0+\beta1X1+\beta2X2+\beta3X3+...+\betanXnY$

Where

Symbol	Meaning
Y	Dependent Variable (Target)
β_0	Intercept (Constant)
$\beta_1, \beta_2, \beta_3...$	Coefficients (Weights)
$X_1, X_2, X_3...$	Independent Variables (Features)

Difference Between Simple and Multiple Linear Regression

Simple Linear Regression	Multiple Linear Regression
One Independent Variable	Two or More Independent Variables
One Predictor	Multiple Predictors
Simpler Model	More Accurate for Real-world Problems
Example: Salary from CGPA	Example: Salary from CGPA, Skills, Experience

---

Real-Life Example 1 – House Price Prediction

Suppose you want to predict the price of a house.

Can the price be predicted using **only the area**?

No.

Many other factors affect the house price.

Input Features

- Area
- Number of Bedrooms

- Number of Bathrooms
- Parking Space
- Distance from City
- House Age

Output

- House Price

Dataset

Area (sq.ft.)	Bedrooms	Bathrooms	Parking	House Age	Price (Lakhs)
1200	2	2	1	8	45
1500	3	2	1	6	58
1800	3	3	2	4	72
2200	4	3	2	2	90
2500	4	4	2	1	110

Here,

Independent Variables (X)

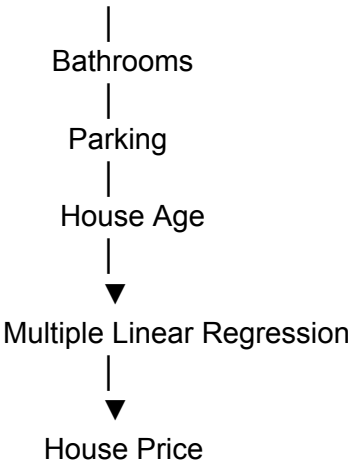
- Area
- Bedrooms
- Bathrooms
- Parking
- House Age

Dependent Variable (Y)

- House Price

Visual Representation

Area
|
Bedrooms



Real-Life Example 2 – Student Placement Prediction

A company wants to predict the salary package of students.

The salary does not depend only on CGPA.

Other factors also matter.

Dataset

CGPA	Aptitude Score	Communication	Experience (Months)	Package (LPA)
6.8	65	6	0	3.2
7.4	72	7	2	4.5
8.0	81	8	5	6.0
8.5	90	9	8	8.1
9.1	96	10	12	10.5

Here,

Input Features

- CGPA

- Aptitude Score
- Communication Skills
- Experience

Target

- Salary Package

Mathematical Example

Suppose the regression equation is

Package=1.2+0.5(CGPTA)+0.02(Aptitude)+0.30(Experience)Package = 1.2 + 0.5(CGPTA) + 0.02(Aptitude) + 0.30(Experience)Package=1.2+0.5(CGPTA)+0.02(Aptitude)+0.30(Experience)

A student has

- CGPA = 8
- Aptitude = 80
- Experience = 6 months

Substitute the values:

Package = 1.2 + (0.5 × 8) + (0.02 × 80) + (0.30 × 6)

Package = 1.2 + 4 + 1.6 + 1.8

Package = 8.6 LPA

Predicted Salary Package = 8.6 LPA

Real-Life Example 3 – Car Price Prediction

Suppose we want to predict the selling price of a used car.

Input Features

- Manufacturing Year
- Present Price

- Kilometers Driven
- Fuel Type
- Seller Type
- Transmission
- Number of Owners

Target

- Selling Price

Example Dataset

Year	Present Price	Driven (km)	Fuel	Transmission	Selling Price
2015	8.5	42000	Petrol	Manual	5.2
2018	12.3	18000	Diesel	Automatic	9.8
2019	15.0	15000	Diesel	Automatic	12.5
2016	9.2	52000	Petrol	Manual	6.1

This is a perfect example of **Multiple Linear Regression** because multiple features influence the car price.

Why Do We Need Multiple Linear Regression?

Suppose we predict house price using only area.

Area = 1500 sq.ft.

Predicted Price = ₹50 Lakhs

But,

Another house with the same area has

- Better Location
- More Bedrooms

- Modern Interior

Actual Price = ₹75 Lakhs

The prediction becomes inaccurate because only one feature was considered.

Therefore, we use Multiple Linear Regression.

Advantages

- Higher prediction accuracy
- Uses multiple input features
- Works well for real-world datasets
- Easy to interpret
- Widely used in business analytics

Limitations

- Sensitive to outliers
- Assumes a linear relationship
- Performance decreases if irrelevant features are included
- Suffers from **multicollinearity** (highly correlated input features)

Assumptions of Multiple Linear Regression

To get accurate predictions, Multiple Linear Regression assumes:

1. Linear Relationship

The relationship between input and output should be approximately linear.

---

2. No Multicollinearity

Independent variables should not be highly correlated with each other.

Bad Example

Height (cm)
Height (feet)

Both represent the same information.

---

3. Independence of Errors

The prediction errors should be independent of one another.

---

4. Homoscedasticity

The variance of errors should remain constant across all observations.

---

5. Normally Distributed Errors

Residual (error) values should follow a normal distribution.

Applications of Multiple Linear Regression

- House Price Prediction
- Salary Prediction
- Sales Forecasting
- Stock Price Analysis
- Weather Forecasting
- Medical Cost Prediction
- Crop Yield Prediction
- Energy Consumption Prediction
- Insurance Premium Prediction
- Business Revenue Prediction

Python Implementation using Scikit-Learn

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Load dataset
data = pd.read_csv("house_price.csv")

# Independent variables
X = data[['Area', 'Bedrooms', 'Bathrooms', 'Parking']]

# Dependent variable
y = data['Price']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
  X, y,
  test_size=0.2,
  random_state=42
)

# Create model
model = LinearRegression()

# Train model
model.fit(X_train, y_train)

# Predict
prediction = model.predict(X_test)

print(prediction)
```

Simple vs Multiple Linear Regression

Feature	Simple Linear Regression	Multiple Linear Regression

Number of Input Features	1	2 or More
Equation	Y=mX+cY = mX + cY=mX+c	Y=β0+β1X1+β2X2+⋯+βnXnY
Complexity	Low	Medium
Prediction Accuracy	Lower	Higher
Real-world Usage	Limited	Very Common

Polynomial Regression

Introduction

In **Simple Linear Regression**, we assume that the relationship between the input feature (X) and the output (Y) is **linear**, meaning the data follows a straight-line pattern.

However, many real-world datasets do **not** follow a straight line. Instead, they follow a **curve**.

In such situations, **Polynomial Regression** is used.

Definition

Polynomial Regression is a supervised machine learning algorithm used to model the relationship between the independent variable(s) and the dependent variable when the relationship is **non-linear** (curved).

Although the graph is curved, Polynomial Regression is still considered a **Linear Regression model**, because the model remains **linear in its coefficients (parameters)**.

Why Do We Need Polynomial Regression?

Suppose we are predicting the salary of an employee based on years of experience.

Dataset

	Experience (Years)	Salary (Lakhs)
1		3
2		4
3		6
4		9
5		13
6		18
7		24

The salary does not increase by a constant amount every year.

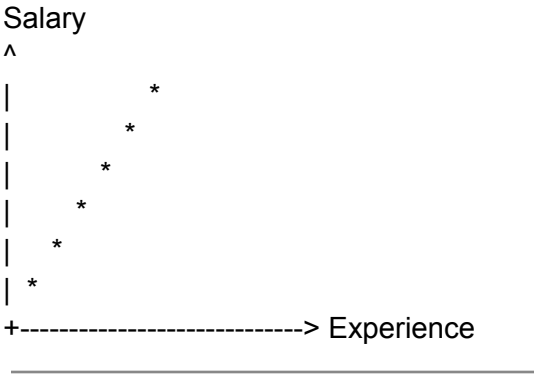
- A **straight line** cannot fit this data accurately.
- A **curve** represents the relationship much better.

Therefore, we use **Polynomial Regression**.

Linear Regression vs Polynomial Regression

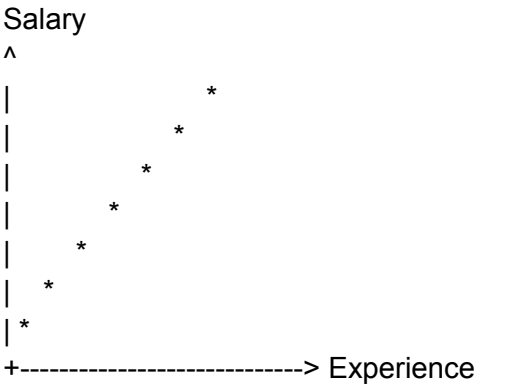
Linear Regression

The relationship is a straight line.



Polynomial Regression

The relationship is curved.



Mathematical Equation

Simple Linear Regression

$Y=mX+cY = mX + cY=mX+c$

Polynomial Regression (Degree 2)

$Y=\beta0+\beta1X+\beta2X2Y$

Polynomial Regression (Degree 3)

$Y=\beta0+\beta1X+\beta2X2+\beta3X3Y =$

General Formula

$Y=\beta0+\beta1X+\beta2X2+\beta3X3+\cdots+\beta nXnY$

Where

Symbol	Meaning
Y	Predicted Output
X	Independent Variable
β_0	Intercept
$\beta_1, \beta_2, \beta_3...$	Coefficients
$X^2, X^3...$	Polynomial Features

How Polynomial Regression Works

Instead of using only **X**, Polynomial Regression creates additional features such as:

Original Feature

X

Converted Features

X
X^2
X^3
X^4

These new features help the model fit curved data.

Example 1 – Student Performance

Suppose we want to predict exam marks based on study hours.

Dataset

Study Hours	Marks
1	20
2	35
3	48
4	58
5	66
6	72
7	75

The marks increase rapidly at first but later begin to level off.

This forms a curved relationship.

Polynomial Regression provides a better fit than Simple Linear Regression.

Example 2 – Salary Prediction

Experience	Salary (Lakhs)
1	3
2	4
3	6
4	9
5	13
6	18
7	24

Suppose the trained model is

Salary=2+0.5X+0.3X2Salary = 2 + 0.5X + 0.3X^2Salary=2+0.5X+0.3X2

If

Experience = 5 Years

Then

Salary = 2 + (0.5 × 5) + (0.3 × 25)

Salary = 2 + 2.5 + 7.5

Salary = 12 Lakhs

Predicted Salary = **12 Lakhs**

Example 3 – Crop Yield Prediction

A farmer wants to predict crop yield using fertiliser quantity.

Fertiliser (kg)	Yield (tons)
10	5
20	9
30	14
40	17
50	18
60	17

Initially, increasing fertiliser increases crop yield.

After a certain point, adding more fertiliser reduces the yield.

This creates a curved relationship.

Polynomial Regression is suitable for this dataset.

Example 4 – Car Speed and Fuel Consumption

Speed (km/h)	Fuel Consumption (L/hr)
20	3
40	4
60	5
80	6
100	8
120	11

Fuel consumption does not increase linearly.

The graph becomes curved.

Polynomial Regression gives better predictions.

Degree of Polynomial

The **degree** determines the complexity of the curve.

Degree = 1

Straight Line

$Y = \beta_0 + \beta_1 X$

Equivalent to Linear Regression.

Degree = 2

Quadratic Curve

$Y = \beta_0 + \beta_1 X + \beta_2 X^2$

Suitable for U-shaped or inverted U-shaped data.

Degree = 3

Cubic Curve

$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3$

Can model more complex relationships.

Degree = 4 or Higher

Can fit highly complex datasets but increases the risk of **overfitting**.

Choosing the Degree

Degree	Suitable For
1	Linear data
2	Slightly curved data
3	Moderately curved data
4+	Complex data (may overfit)

Steps in Polynomial Regression

1. Load the dataset.
2. Select input feature(s) and target variable.
3. Create polynomial features (X^2, X^3, ...).
4. Split the data into training and testing sets.
5. Train a Linear Regression model using the polynomial features.

6. Predict new values.
7. Evaluate the model.

Advantages

- Captures non-linear relationships.
- More accurate than Linear Regression for curved datasets.
- Easy to implement using Scikit-Learn.
- Works well for moderate non-linear problems.

Disadvantages

- Can overfit when the degree is too high.
- Sensitive to outliers.
- Harder to interpret than Simple Linear Regression.
- Computational cost increases with higher-degree polynomials.

Applications

- Salary Prediction
- House Price Prediction
- Crop Yield Prediction
- Population Growth Analysis
- Weather Forecasting
- Sales Forecasting
- Medical Data Analysis
- Stock Market Trend Analysis

Python Implementation (Scikit-Learn)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Load dataset
data = pd.read_csv("salary.csv")

# Independent Variable
X = data[['Experience']]

# Dependent Variable
y = data['Salary']

# Create Polynomial Features
poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
  X_poly,
  y,
  test_size=0.2,
  random_state=42
)

# Train Model
model = LinearRegression()

model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)

print(prediction)
```

Linear Regression vs Polynomial Regression

Feature	Linear Regression	Polynomial Regression
Relationship	Linear	Non-linear (Curved)
Graph	Straight Line	Curve
Formula	$Y=mX+cY = mX + cY=mX+c$	$Y=\beta0+\beta1X+\beta2X2+\cdots Y$
Accuracy	Good for linear data	Better for curved data
Complexity	Low	Medium to High
Risk of Overfitting	Low	Higher for large polynomial degrees

Tab 4

What is a Cost Function?

A **Cost Function** is a mathematical function that measures **how well or how poorly a machine learning model is performing**.

It calculates the difference between the **actual output** and the **predicted output**.

- **Smaller Cost → Better Model**
- **Larger Cost → Poor Model**

Definition

> A Cost Function is a mathematical function used to measure the error between the actual values and the predicted values of a machine learning model.

Why Do We Need a Cost Function?

When we train a machine learning model, its predictions are not perfect initially.

The model needs a way to determine:

- How much error it is making.
- Whether it is improving after each training iteration.
- How to adjust its parameters (weights and bias).

The Cost Function provides this information.

Real-Life Analogy

Imagine a student appearing for a mathematics exam.

- Total Marks = 100
- Student Scores = 45

The teacher compares:

- Expected Marks = 100
- Obtained Marks = 45

Difference = 55 marks

Similarly, a machine learning model compares:

- Actual Value
- Predicted Value

The difference between them is called the **error**.

The Cost Function measures the total error made by the model.

Example

Suppose we are predicting house prices.

House	Actual Price (Lakhs)	Predicted Price (Lakhs)
A	50	48
B	65	70
C	80	78
D	95	90

Error Calculation

Actual	Predicted	Error
50	48	2
65	70	-5
80	78	2
95	90	5

The Cost Function combines all these errors into a **single numerical value**.

Error vs Cost Function

Error	Cost Function
Difference for one data point	Average error for the entire dataset
Single prediction	Entire model
Small calculation	Overall performance

Types of Cost Functions

The choice of the cost function depends on the machine learning problem.

Regression Problems

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

Classification Problems

- Binary Cross-Entropy
- Categorical Cross-Entropy

Mean Squared Error (MSE)

Mean Squared Error (MSE)

For Linear Regression, the most commonly used cost function is the **Mean Squared Error (MSE)**.

Formula

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where

Symbol	Meaning
$J(\theta)$	Cost Function
n	Number of Training Samples
y_i	Actual Value
$\hat{y}_i$	Predicted Value

Understanding the Formula

The Mean Squared Error (MSE) computes the average of the squared differences between the actual and predicted values.

Key steps:

1. Calculate the error for each prediction: **Actual − Predicted**.
2. Square each error (to remove negative signs and penalize larger errors).
3. Add all the squared errors.
4. Divide by the total number of samples.

Example

Suppose we have the following data:

Actual (Y)	Predicted (Ŷ)
10	9
15	16
20	18

Step 1: Calculate Error

Actual	Predicted	Error
10	9	1
15	16	-1
20	18	2

Step 2: Square the Errors

Error	Squared Error
1	1
-1	1
2	4

Step 3: Find the Average

$$MSE = \frac{1 + 1 + 4}{3}$$

$$MSE = \frac{6}{3} = 2$$

Cost (MSE) = 2

Why Do We Square the Error?

Suppose the errors are:

Error 1 = -5

Error 2 = +5

If we simply add them:

-5 + 5 = 0

This incorrectly suggests that the model made no error.

By squaring the errors:

$(-5)^2 = 25$

$(+5)^2 = 25$

Both errors contribute positively to the total cost, preventing positive and negative errors from cancelling each other.

Why Is Lower Cost Better?

Suppose two models predict the same dataset.

Model A

Cost = 2

Model B

Cost = 25

Model A is better because its predictions are closer to the actual values.

Goal of Machine Learning:

Minimise the Cost Function

Relationship Between Cost Function and Gradient Descent

The Cost Function tells us **how much error** the model has.

Gradient Descent tells us **how to reduce that error** by updating the model's parameters.

Training Data

Prediction

Cost Function

Gradient Descent

Update Weights

Better Prediction

This process repeats until the cost becomes as small as possible.

Cost Function During Training

Initially

Cost = 120

After several iterations

Cost = 70

Later

Cost = 25

Finally

Cost = 5

The model gradually improves because the cost decreases with each iteration.

Advantages of Cost Function

- Measures model performance.
- Helps compare different models.
- Guides the optimization process.
- Essential for Gradient Descent.
- Indicates whether the model is learning effectively.

Limitations

- Different problems require different cost functions.
- Sensitive to outliers (especially MSE).
- May converge to local minima in some optimization problems.

Applications

- Linear Regression
- Polynomial Regression
- Logistic Regression
- Neural Networks
- Deep Learning
- Recommendation Systems
- Stock Price Prediction
- House Price Prediction

Python Example

```
from sklearn.metrics import mean_squared_error

actual = [10, 15, 20]
predicted = [9, 16, 18]

mse = mean_squared_error(actual, predicted)

print("Cost (MSE):", mse)
```

Output

```
Cost (MSE): 2.0
```

Cost Function vs Loss Function

Cost Function	Loss Function
Average error over the entire dataset	Error for a single training example
Used to evaluate the overall model	Used to measure the error of one prediction
Example: Mean Squared Error (MSE) over all samples	Squared Error for one sample

---

Tab 5

Regression Evaluation Metrics: R^2 Score & Adjusted R^2 Score (Complete Notes)

Introduction

After training a regression model, we need to evaluate how well it predicts the target values.

Common evaluation metrics include:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- **R^2 Score (Coefficient of Determination)**
- **Adjusted R^2 Score**

Among these, **R^2 Score** and **Adjusted R^2 Score** are the most widely used to measure the performance of regression models.

What is the R^2 Score?

The **R^2 Score**, also known as the **Coefficient of Determination**, measures how well the regression model explains the variation in the dependent (target) variable.

It indicates **how much of the variation in the output is explained by the input features**.

Definition

> The R^2 Score is a statistical metric that measures the proportion of variance in the dependent variable that is explained by the independent variables.

Why Do We Need the R^2 Score?

Suppose we build a model to predict house prices.

After training the model, we want to know:

- Is the model accurate?
- How much information from the data has the model learned?
- Can the model explain most of the variation in house prices?

The **R² Score** answers these questions.

Formula of R² Score

$$R^2 = 1 - \frac{SS_{Residual}}{SS_{Total}}$$

Where

Symbol	Meaning
R^2	Coefficient of Determination
$SS_{Residual}$	Sum of Squared Errors (Unexplained Variation)
SS_{Total}	Total Variation in the Data

Understanding the Formula

Total Sum of Squares (SST)

Measures the total variation in the actual data.

$$SS_{Total} = \sum (y_i - \bar{y})^2$$

Where:

- y_i = Actual value
- $\bar{y}$ = Mean of the actual values

Residual Sum of Squares (SSR)

Measures the error between actual and predicted values.

$$SS_{Residual} = \sum (y_i - \hat{y}_i)^2$$

Where:

- y_i = Actual value
- $\hat{y}_i$ = Predicted value

Interpretation of the R^2 Score

R^2 Score	Interpretation
1.0	Perfect prediction
0.95	Excellent model
0.80	Very good model
0.60	Average model
0.30	Poor model
0.0	No predictive ability
< 0	Model performs worse than predicting the mean

Example

Suppose the actual and predicted house prices are:

Actual	Predicted
50	49
60	62
70	69

80	78
90	91

After calculation,

R^2 Score = 0.94

This means:

94% of the variation in house prices is explained by the regression model.

Only **6% of the variation remains unexplained**.

Real-Life Analogy

Imagine a teacher predicts students' marks based on study hours.

If the **R^2 Score = 0.90**

It means:

> **90% of the variation in students' marks is explained by study hours.**

The remaining **10%** may be due to:

- Health
- Sleep
- Exam difficulty
- Concentration
- Other factors

Advantages of the R^2 Score

- Easy to interpret.
- Measures goodness of fit.
- Widely used in regression analysis.
- Helps compare different regression models.

Limitations of the R^2 Score

The R^2 Score always increases (or remains the same) when additional features are added to the model, even if those features are irrelevant.

This can make the model appear better than it actually is.

Example:

Suppose we predict house prices.

Useful features:

- Area
- Bedrooms
- Location

Irrelevant feature:

- House Owner's Favorite Color

Even after adding this useless feature,

Old $R^2 = 0.82$

New $R^2 = 0.84$

The R^2 Score increases even though the new feature has no real predictive value.

This is why we use the **Adjusted R^2 Score**.

What is the Adjusted R^2 Score?

The **Adjusted R^2 Score** is an improved version of the R^2 Score.

It adjusts the score based on:

- Number of independent variables (features)
- Number of observations (samples)

Unlike the R² Score, it **penalises unnecessary features**.

Definition

> Adjusted R² measures the goodness of fit while considering the number of predictor variables used in the regression model.

Formula of Adjusted R²

$$Adjusted\ R^2 = 1 - \left(1 - R^2\right) \times \frac{n-1}{n-p-1}$$

Where

Symbol	Meaning
R^2	R² Score
n	Number of observations
p	Number of independent variables

Why Do We Need Adjusted R²?

Suppose we have two models.

Model A

Features:

- Area
- Bedrooms

R² = 0.88

Adjusted R² = 0.87

Model B

Features:

- Area
- Bedrooms
- Parking
- Garden
- House Color
- Owner's Name

$R^2 = 0.92$

Adjusted $R^2 = 0.84$

Although **Model B** has a higher R^2 Score, its Adjusted R^2 Score is lower because many added features are not useful.

Therefore, **Model A is the better model**.

Comparison Between R^2 and Adjusted R^2

Feature	R^2 Score	Adjusted R^2 Score
Measures Model Fit	Yes	Yes
Considers Number of Features	No	Yes
Increases When Features Are Added	Always	Only if the new feature improves the model
Penalizes Irrelevant Features	No	Yes
Best For	Simple Regression	Multiple Regression

Example

Suppose we build three regression models.

Model	Features	R^2	Adjusted R^2
Model A	Area	0.82	0.82
Model B	Area, Bedrooms	0.90	0.89
Model C	Area, Bedrooms, Color, Owner Name	0.93	0.85

Although Model C has the highest R^2 Score, its Adjusted R^2 Score decreases because the additional features do not meaningfully improve the model.

Therefore, **Model B** is preferred.

When Should We Use R^2?

Use the R^2 Score when:

- Comparing simple regression models.
- The number of features is small.
- You only need to know how well the model fits the data.

When Should We Use Adjusted R^2?

Use the Adjusted R^2 Score when:

- Working with **Multiple Linear Regression**.
- Comparing models with different numbers of input features.
- Performing feature selection.
- Detecting unnecessary predictors.

Python Example

```
from sklearn.metrics import r2_score
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
import pandas as pd

# Load dataset
data = pd.read_csv("house_price.csv")

# Features
X = data[['Area', 'Bedrooms', 'Bathrooms']]

# Target
y = data['Price']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
   X, y,
   test_size=0.2,
   random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# R² Score
r2 = r2_score(y_test, y_pred)

print("R² Score:", r2)
```

Sample Output

R² Score: 0.91

This indicates that **91% of the variation in house prices is explained by the model**.

Applications

- House Price Prediction
- Salary Prediction
- Stock Price Analysis

- Medical Cost Prediction
- Weather Forecasting
- Sales Prediction
- Crop Yield Prediction
- Demand Forecasting

Key Differences

Feature	R^2 Score	Adjusted R^2 Score
Full Name	Coefficient of Determination	Adjusted Coefficient of Determination
Range	Can be ≤ 1 (typically 0 to 1 for useful models)	Can be ≤ 1 (typically 0 to 1)
Penalizes Extra Features	No	Yes
Best for Multiple Regression	No	Yes
Used for Feature Selection	No	Yes
More Reliable for Comparing Models	No	Yes

Tab 6

How to Find the Best Fit Line in Linear Regression

Introduction

In **Linear Regression**, the objective is to draw a straight line that best represents the relationship between the independent variable (X) and the dependent variable (Y).

This line is called the **Best Fit Line** or **Regression Line**.

The Best Fit Line minimizes the prediction error between the actual values and the predicted values.

What is the Best Fit Line?

A **Best Fit Line** is the straight line that passes through the data points in such a way that the **overall prediction error is as small as possible**.

Definition

> The Best Fit Line is the line that minimizes the total prediction error between the actual values and the predicted values using the Least Squares Method.

Mathematical Equation of the Best Fit Line

The equation of a straight line is:

$Y=mX+cY = mX + cY=mX+c$

Where:

Symbol	Meaning
Y	Predicted Output
X	Input Feature
m	Slope of the Line
c	Y-intercept

Understanding the Equation

Suppose the regression equation is:

$Y=2X+3Y = 2X + 3Y=2X+3$

If

$X = 5$

Then

$Y = 2 \times 5 + 3$
$Y = 13$

So, when the input is **5**, the predicted output is **13**.

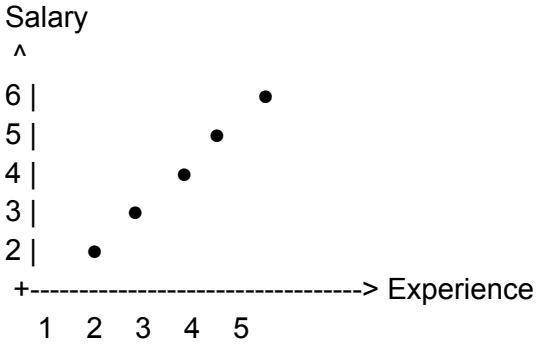
Example Dataset

Suppose we want to predict salary based on years of experience.

Experience (Years)	Salary (Lakhs)
1	2
2	3
3	5

4	4
5	6

Scatter Plot

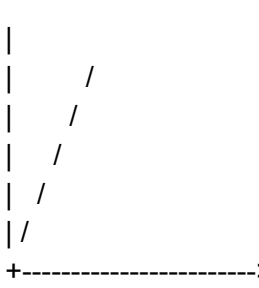


Many straight lines can be drawn through these points.

But **which one is the best?**

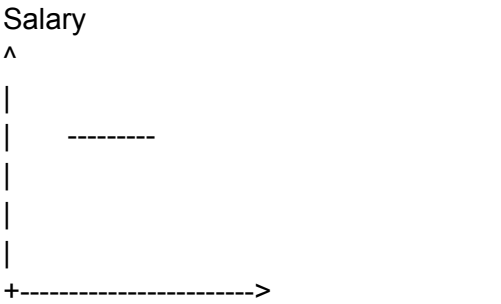
Possible Lines

Line A



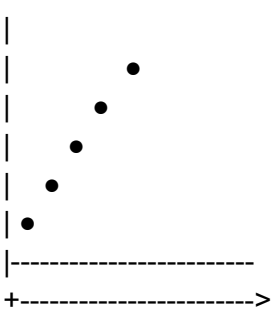
Large prediction error.

Line B



Does not represent the trend.

Line C (Best Fit Line)



Most data points lie close to the line.

Prediction error is minimum.

How is the Best Fit Line Found?

The Best Fit Line is obtained using the **Least Squares Method**.

Instead of minimizing the simple errors,

Actual − Predicted

we minimize the **sum of squared errors**.

Step 1: Predict Values

Suppose the regression equation is

$Y=X+1Y = X + 1Y=X+1$

Dataset

X	Actual Y	Predicted Y
1	2	2
2	3	3
3	5	4
4	4	5
5	6	6

Step 2: Calculate Errors

Actual	Predicted	Error
2	2	0
3	3	0
5	4	1
4	5	-1
6	6	0

Step 3: Square the Errors

Error	Error²
0	0

0	0
1	1
-1	1
0	0

Total Squared Error

0 + 0 + 1 + 1 + 0 = 2

Least Squares Method

The Best Fit Line is the line with the **smallest Sum of Squared Errors (SSE)**.

Suppose we test three different lines.

Line	Sum of Squared Errors
Line A	20
Line B	9
Line C	2

Since **Line C** has the smallest error,

Line C is selected as the Best Fit Line.

Why Do We Square the Errors?

Suppose

Error = +5

Error = -5

Adding them directly gives

5 + (-5) = 0

This incorrectly suggests there is no error.

Squaring avoids this problem.

$(+5)^2 = 25$

$(-5)^2 = 25$

Both errors contribute positively to the total error.

Role of Cost Function

The Cost Function measures how good the Best Fit Line is.

For Linear Regression,

$$J = \frac{1}{n}\sum(y - \hat{y})^2$$

- Smaller Cost → Better Fit
- Larger Cost → Poor Fit

The objective is to minimise the Cost Function.

Role of Gradient Descent

Initially,

The slope (**m**) and intercept (**c**) are random.

Example

m = 0.5

c = 10

Prediction is poor.

Gradient Descent gradually updates **m** and **c**.

Iteration 1

m = 0.7

c = 8

↓

Iteration 2

m = 1.1

c = 5

↓

Iteration 3

m = 1.4

c = 2

↓

Finally

m = 1.6

c = 1

Now the Cost Function is minimum.

The line becomes the **Best Fit Line**.

Characteristics of the Best Fit Line

- Passes as close as possible to all data points.
- Minimizes the Sum of Squared Errors (SSE).
- Produces the smallest Cost Function.
- Gives the best prediction for unseen data.
- Represents the overall trend of the data.

Real-Life Example

Suppose a company wants to predict an employee's salary based on years of experience.

Dataset

Experience	Salary (Lakhs)
1	2
2	3
3	4.5
4	5
5	6.5

The Best Fit Line learns the relationship between experience and salary.

For a new employee with:

Experience = 6 Years

The model predicts

Salary ≈ 7.2 Lakhs

Python Example

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Independent Variable
X = np.array([1, 2, 3, 4, 5]).reshape(-1,1)

# Dependent Variable
y = np.array([2, 3, 5, 4, 6])

# Create Model
model = LinearRegression()

# Train Model
model.fit(X, y)

# Best Fit Line Parameters
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
```

Output (Example)

Slope = 0.9

Intercept = 1.2

Regression Equation

Y=0.9X+1.2Y = 0.9X + 1.2Y=0.9X+1.2

Applications

- House Price Prediction
- Salary Prediction
- Sales Forecasting
- Weather Forecasting
- Stock Price Prediction
- Demand Forecasting

Tab 7

L1 (Lasso) and L2 (Ridge) Regularisation

Introduction

In Machine Learning, a regression model may perform very well on the training data but poorly on unseen (test) data. This happens because the model **memorises the training data instead of learning the underlying pattern**. This problem is called **Overfitting**.

To reduce overfitting and improve the model's ability to generalise to new data, we use a technique called **Regularization**.

What is Regularisation?

Regularisation is a technique used in Machine Learning to reduce model complexity by adding a **penalty term** to the cost function. This discourages the model from assigning excessively large values to the coefficients (weights).

Definition

> Regularisation is a technique that prevents overfitting by adding a penalty to the model's cost function, encouraging simpler models that generalise better to unseen data.

Why Do We Need Regularisation?

Suppose we want to predict house prices using **100 input features**.

Many of these features may be unnecessary, such as:

- House owner's favorite color
- Owner's lucky number
- Wallpaper color

If all features are used, the model may memorise the training data and fail to perform well on new data.

Regularisation helps by reducing the influence of unimportant features.

Without Regularization

Training Accuracy = 99%

Testing Accuracy = 72%

The model has **overfitted** the training data.

With Regularization

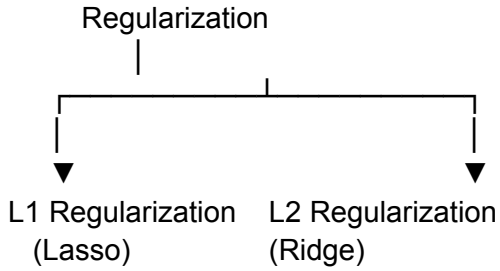
Training Accuracy = 95%

Testing Accuracy = 93%

The model generalises better to unseen data.

Types of Regularisation

There are two major regularisation techniques used in regression:



L1 Regularisation (Lasso Regression)

Definition

L1 Regularisation, also known as **Lasso Regression (Least Absolute Shrinkage and Selection Operator)**, adds the **absolute values of the coefficients** to the cost function.

It can reduce some coefficients **exactly to zero**, thereby performing **automatic feature selection**.

Cost Function

$$J(\theta) = MSE + \lambda \sum_{j=1}^{n} |\beta_j|$$

Where

Symbol	Meaning
J(θ)	Total Cost
MSE	Mean Squared Error
λ	Regularization Parameter
βj	Model Coefficients

How Lasso Works

Suppose a model has the following coefficients:

Feature	Coefficient
Area	5.2
Bedrooms	3.8
Bathrooms	2.5
Parking	0.1
Owner Name	0.05
Wall Color	0.02

After applying Lasso:

Feature	New Coefficient
Area	5.1
Bedrooms	3.6
Bathrooms	2.4
Parking	0
Owner Name	0
Wall Color	0

Lasso removes the irrelevant features by assigning their coefficients a value of **zero**.

Advantages of Lasso

- Performs automatic feature selection.
- Reduces overfitting.
- Produces simpler and more interpretable models.

- Works well when many features are irrelevant.

Disadvantages of Lasso

- Can remove useful features if they are highly correlated.
- Less stable when features are strongly correlated.
- May not perform well when all features are important.

Applications of Lasso

- Feature Selection
- Medical Diagnosis
- Gene Expression Analysis
- Text Classification
- Financial Modeling

L2 Regularisation (Ridge Regression)

Definition

L2 Regularisation, also known as **Ridge Regression**, adds the **square of the coefficients** to the cost function.

Unlike Lasso, Ridge **does not make coefficients exactly zero**. Instead, it reduces their magnitude, keeping all features in the model.

Cost Function

$$J(\theta) = MSE + \lambda \sum_{j=1}^{n} \beta_j^2$$

Where

Symbol	Meaning
J(θ)	Total Cost
MSE	Mean Squared Error
λ	Regularization Parameter
βj	Model Coefficients

How Ridge Works

Original coefficients:

Feature	Coefficient
Area	5.2
Bedrooms	3.8
Bathrooms	2.5
Parking	0.5
Owner Name	0.3
Wall Color	0.2

After Ridge:

Feature	New Coefficient
Area	4.9
Bedrooms	3.5
Bathrooms	2.3

Parking	0.4
Owner Name	0.2
Wall Color	0.1

Notice that **all coefficients are reduced**, but **none become zero**.

Advantages of Ridge

- Reduces overfitting.
- Handles multicollinearity effectively.
- Retains all input features.
- Produces stable models.

Disadvantages of Ridge

- Does not perform feature selection.
- Keeps irrelevant features in the model.
- Harder to interpret compared to Lasso.

Applications of Ridge

- House Price Prediction
- Sales Forecasting
- Weather Prediction
- Stock Price Analysis
- Economic Forecasting

Understanding the Regularisation Parameter (λ)

The symbol **λ (lambda)** controls the strength of regularisation.

λ = 0

No Regularization

The model behaves like ordinary Linear Regression.

Small λ

Slight Regularization

Coefficients are reduced slightly.

Large λ

Strong Regularization

Coefficients become very small.

If λ is too large, the model may **underfit** because it becomes too simple.

Graphical Understanding

Without Regularization

Very Complex Curve

The model overfits the training data.

With Regularization

Smooth Curve

The model captures the overall trend and generalises better to new data.

Lasso vs Ridge Example

Suppose we have six features.

Feature	Original	Lasso	Ridge
Area	5.2	5.0	5.0
Bedrooms	4.5	4.2	4.3
Bathrooms	3.0	2.8	2.9
Parking	0.4	0	0.3
Owner Name	0.2	0	0.1
Wall Color	0.1	0	0.05

Observation:

- Lasso removes unnecessary features.
- Ridge keeps all features but reduces their impact.

Difference Between Lasso and Ridge

Feature	Lasso (L1)	Ridge (L2)
Full Name	Least Absolute Shrinkage and Selection Operator	Ridge Regression
Penalty	Absolute Value of Coefficients	Square of Coefficients
Formula	(MSE+\lambda\sum	\beta
Feature Selection	Yes	No

Coefficients Become Zero	Yes	No
Handles Multicollinearity	Good	Excellent
Model Simplicity	High	Moderate
Best When	Many irrelevant features exist	Most features are useful

---

When Should We Use Lasso?

Use **Lasso Regression** when:

- There are many input features.
- Some features are irrelevant.
- Feature selection is required.
- You want a simpler, more interpretable model.

When Should We Use Ridge?

Use **Ridge Regression** when:

- Most features are important.
- Features are highly correlated (multicollinearity).
- You want to retain all variables while reducing overfitting.

Python Example

Lasso Regression

```
from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)
```

```
model.fit(X_train, y_train)

prediction = model.predict(X_test)
```

Ridge Regression

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=0.1)
model.fit(X_train, y_train)

prediction = model.predict(X_test)
```

Applications of Regularisation

- House Price Prediction
- Medical Diagnosis
- Sales Forecasting
- Credit Risk Analysis
- Weather Prediction
- Stock Market Analysis
- Customer Churn Prediction
- Text Classification

Key Points to Remember

- **Regularisation** reduces overfitting by adding a penalty to the cost function.
- **L1 Regularisation (Lasso)** uses the **absolute values** of coefficients and can reduce some coefficients to **zero**, effectively performing feature selection.
- **L2 Regularisation (Ridge)** uses the **squares** of coefficients, reducing their magnitude without eliminating features.
- The **regularisation parameter (λ)** controls the strength of the penalty. A larger λ increases regularisation, while λ = 0 corresponds to standard Linear Regression.
- **Lasso** is preferred when feature selection is important, whereas **Ridge** is preferred when most features are useful, and multicollinearity is present.

Practical: L1 (Lasso) and L2 (Ridge) Regularisation using Python (Scikit-Learn)

This practical demonstrates how to implement **Lasso Regression (L1 Regularisation)** and **Ridge Regression (L2 Regularisation)** using Python and Scikit-Learn.

Aim

To implement **Lasso Regression** and **Ridge Regression** for predicting house prices and compare their performance.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Scikit-Learn
- Pandas
- NumPy
- Matplotlib

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso

from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

Step 2: Create Sample Dataset

```
data = {
  'Area':[1000,1200,1500,1800,2000,2300,2500,2700,3000,3200],
  'Bedrooms':[2,2,3,3,4,4,4,5,5,5],
  'Bathrooms':[1,2,2,2,3,3,3,4,4,4],
  'Parking':[1,1,2,2,2,3,3,3,4,4],
  'Price':[30,35,45,55,65,75,85,95,110,120]
}

df = pd.DataFrame(data)

print(df)
```

Output

Area	Bedrooms	Bathrooms	Parking	Price
1000	2	1	1	30
1200	2	2	1	35
1500	3	2	2	45
...	...	...	...	...

Step 3: Select Features and Target

```
X = df[['Area','Bedrooms','Bathrooms',' Parking']]

y = df['Price']
```

Step 4: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.2,
  random_state=42
```

)

Training Data = 80%

Testing Data = 20%

Practical 1: Linear Regression (Without Regularisation)

```
linear = LinearRegression()

linear.fit(X_train,y_train)

y_pred = linear.predict(X_test)

print("Linear Regression")
print("R2 Score :",r2_score(y_test,y_pred))
print("MSE :",mean_squared_error(y_test,y_pred))
```

Practical 2: Ridge Regression (L2)

```
ridge = Ridge(alpha=1.0)

ridge.fit(X_train,y_train)

ridge_pred = ridge.predict(X_test)

print("Ridge Regression")

print("R2 Score :",r2_score(y_test,ridge_pred))

print("MSE :",mean_squared_error(y_test,ridge_pred))
```

Understanding alpha

Ridge(alpha=0.1)

Ridge(alpha=1)

Ridge(alpha=10)

Ridge(alpha=100)

Higher alpha means

- Stronger Regularization
- Smaller Coefficients

Practical 3: Lasso Regression (L1)

lasso = Lasso(alpha=1.0)

lasso.fit(X_train,y_train)

lasso_pred = lasso.predict(X_test)

print("Lasso Regression")

print("R2 Score :",r2_score(y_test,lasso_pred))

print("MSE :",mean_squared_error(y_test,lasso_pred))

Compare Coefficients

Linear Regression

print(linear.coef_)

Example Output

[0.032 2.5 3.1 1.8]

Ridge Regression

print(ridge.coef_)

Example Output

[0.030 2.3 2.8 1.5]

Notice

All coefficients become smaller.

Lasso Regression

print(lasso.coef_)

Example Output

[0.028 2.2 0.0 1.1]

Notice

Bathroom coefficient becomes **0**

Lasso automatically removed that feature.

Predict New House Price

new_house = [[2100,4,3,2]]

price = ridge.predict(new_house)

print(price)

Output

[69.5]

Predicted Price = **69.5 Lakhs**

Compare All Models

print("Linear Regression R2 :",r2_score(y_test,y_pred))

print("Ridge Regression R2 :",r2_score(y_test,ridge_pred))

print("Lasso Regression R2 :",r2_score(y_test,lasso_pred))

Example Output

Linear Regression R2 : 0.97

Ridge Regression R2 : 0.98

Lasso Regression R2 : 0.96

Visualizing Coefficients

models=['Linear','Ridge','Lasso']

coef1=linear.coef_[0]

coef2=ridge.coef_[0]

coef3=lasso.coef_[0]

plt.bar(models,[coef1,coef2,coef3])

plt.title("Comparison of Area Coefficient")

plt.ylabel("Coefficient Value")

plt.show()

This graph shows how Regularization reduces coefficient values.

Using a Real Dataset

Instead of manually creating data, you can use a CSV file.

df = pd.read_csv("HousePrice.csv")

Then repeat

X = df.drop("Price",axis=1)

y = df["Price"]

Remaining code stays exactly the same.

Complete Workflow

Load Dataset

Data Preprocessing

Train Test Split

Linear Regression

Ridge Regression

Lasso Regression

Prediction

Evaluation

Compare Models

Expected Output

Linear Regression

R2 Score : 0.97

MSE : 4.23

Ridge Regression

R2 Score : 0.98

MSE : 3.85

Lasso Regression

R2 Score : 0.96

MSE : 4.71

Tab 8

Classification in Machine Learning (Complete Notes)

Introduction

In Machine Learning, there are many real-world problems where we need to predict **categories or classes** instead of numerical values.

For example:

- Is an email **Spam** or **Not Spam**?
- Does a patient have **Diabetes** or **No Diabetes**?
- Is the image a **Cat** or a **Dog**?

These problems are solved using **Classification**.

What is Classification?

Classification is a **Supervised Machine Learning** technique used to predict **categorical (discrete) values** based on input features.

Unlike Regression, which predicts numerical values, Classification predicts **labels or categories**.

Definition

> Classification is a supervised learning algorithm that learns from labeled data and predicts the category or class of new, unseen data.

Example

Suppose we want to predict whether a student will **Pass** or **Fail**.

Study Hours	Attendance	Result
2	60%	Fail
4	70%	Pass
5	80%	Pass
1	50%	Fail
6	90%	Pass

Here,

Independent Variables (X)

- Study Hours
- Attendance

Dependent Variable (Y)

- Pass / Fail

Since the output is a **category**, this is a **Classification Problem**.

Why Do We Need Classification?

Many real-world applications require predicting categories instead of numbers.

Examples include:

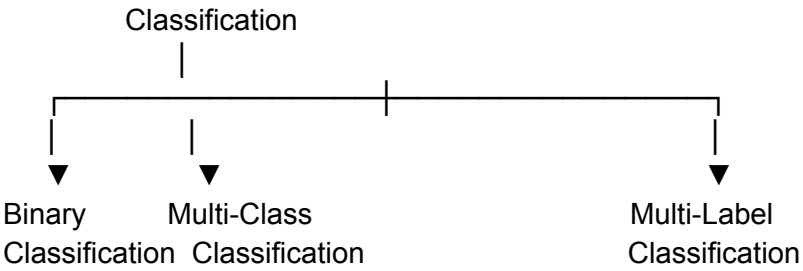
- Email Spam Detection
- Disease Diagnosis
- Face Recognition
- Fraud Detection
- Customer Churn Prediction

- Sentiment Analysis

Classification algorithms solve these problems efficiently.

Types of Classification

Classification problems are mainly divided into three categories.



---

1. Binary Classification

Binary Classification predicts **only two classes**.

Examples

- Yes / No
- True / False
- Spam / Not Spam
- Pass / Fail
- Disease / No Disease

Example Dataset

Age	Fever	Disease
25	Yes	Positive
18	No	Negative
35	Yes	Positive
22	No	Negative

Output:

Positive

Negative

---

2. Multi-Class Classification

Multi-Class Classification predicts **more than two classes**.

Example

Animal Recognition

Input Image

↓

Output

Cat

Dog

Horse

Cow

Example Dataset

Petal Length	Petal Width	Flower
1.4	0.2	Setosa
4.5	1.5	Versicolor
5.8	2.3	Virginica

Output has **three classes**.

---

3. Multi-Label Classification

In Multi-Label Classification, one input can belong to **multiple categories simultaneously**.

Example

Movie Genre Prediction

Movie:

Avengers

Output

Action

Adventure

Science Fiction

One movie belongs to multiple genres.

Regression vs Classification

Regression	Classification
Predicts numerical values	Predicts categories
Output is continuous	Output is discrete
Example: House Price	Example: Spam Detection
Example: Salary	Example: Disease Prediction

Common Classification Algorithms

Several algorithms are available for classification.

1. Logistic Regression

- Used mainly for Binary Classification.
- Predicts probabilities between 0 and 1.
- Simple and fast.

Applications

- Spam Detection
- Disease Prediction

---

2. K-Nearest Neighbours (KNN)

Predicts the class based on the majority class among the nearest neighbors.

Example

If five nearest neighbors are

Cat

Cat

Cat

Dog

Cat

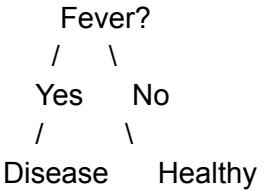
Prediction = **Cat**

---

3. Decision Tree

Uses a tree-like structure.

Example



Easy to understand and visualise.

---

4. Random Forest

Random Forest is a collection of multiple Decision Trees.

The final prediction is based on **majority voting**.

Advantages

- Higher accuracy
- Less overfitting

---

5. Support Vector Machine (SVM)

Finds the **best boundary (hyperplane)** between different classes.

Suitable for

- Image Classification
- Face Recognition
- Medical Diagnosis

---

6. Naïve Bayes

Based on **Bayes' Theorem**.

Widely used for

- Spam Detection
- Text Classification
- Sentiment Analysis

Classification Workflow

Collect Dataset



Data Preprocessing



Train-Test Split



Train Classification Model



Prediction



Model Evaluation

Real-Life Applications

Email Spam Detection

Features

- Email Content
- Sender
- Keywords

Output

Spam

Not Spam

Disease Prediction

Input

- Age
- Blood Pressure

- Sugar Level

Output

Positive

Negative

Face Recognition

Input

Face Image

Output

Person A

Person B

Person C

Loan Approval

Input

- Salary
- Credit Score
- Income

Output

Approved

Rejected

Evaluation Metrics

Unlike Regression, Classification uses different evaluation metrics.

The most common metrics are:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- ROC-AUC Score

Accuracy

Accuracy measures the percentage of correctly classified samples.

Formula

Accuracy=Correct PredictionsTotal PredictionsAccuracy=\frac{Correct\ Predictions}{Total\ Predictions}Accuracy=Total PredictionsCorrect Predictions

Example

100 students

Correct Predictions = 92

Accuracy

92%

Advantages

- Easy to understand.
- Suitable for categorical prediction.
- Works well for many real-world applications.
- Can handle binary and multi-class problems.

Limitations

- Sensitive to imbalanced datasets.
- Some algorithms require feature scaling.
- Performance depends on the quality of the data.

Python Example (Logistic Regression)

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv("diabetes.csv")

# Features
X = df.drop("Outcome",axis=1)

# Target
y = df["Outcome"]

# Split
X_train,X_test,y_train,y_test = train_test_split(
  X,
  y,
  test_size=0.2,
  random_state=42
)

# Create Model
model = LogisticRegression()

# Train
model.fit(X_train,y_train)
```

```
# Predict
prediction=model.predict(X_test)

# Accuracy
print("Accuracy:",accuracy_score(y_test,prediction))
```

Output

Accuracy: 94%

Applications of Classification

- Email Spam Detection
- Disease Diagnosis
- Face Recognition
- Handwriting Recognition
- Fraud Detection
- Customer Churn Prediction
- Loan Approval
- Product Recommendation
- Sentiment Analysis
- Fake News Detection

Summary

- **Classification** is a **Supervised Machine Learning** algorithm.
- It predicts **categorical (discrete)** outputs rather than numerical values.
- The three main types are:
    - **Binary Classification** (two classes)
    - **Multi-Class Classification** (more than two classes)
    - **Multi-Label Classification** (multiple labels for one instance)

- Popular classification algorithms include **Logistic Regression**, **K-Nearest Neighbours (KNN)**, **Decision Tree**, **Random Forest**, **Support Vector Machine (SVM)**, and **Naïve Bayes**.
- Common evaluation metrics are **Accuracy**, **Precision**, **Recall**, **F1-Score**, and **Confusion Matrix**.

Practical: Logistic Regression (Binary Classification) using Python (Scikit-Learn)

Aim

To implement **Logistic Regression** for a **Binary Classification** problem using Python and Scikit-Learn.

Objective

- Load a dataset.
- Split the dataset into training and testing sets.
- Train a Logistic Regression model.
- Predict the class labels.
- Evaluate the model using different performance metrics.

Software Requirements

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Scikit-Learn
- Matplotlib (Optional)

Problem Statement

We want to predict whether a patient has **Diabetes**.

Output Classes

- **0 → No Diabetes**
- **1 → Diabetes**

Since there are only **two classes**, this is a **Binary Classification** problem.

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Step 2: Load Dataset

```
df = pd.read_csv("diabetes.csv")
```

print(df.head())

Example Dataset

Pregnancies	Glucose	BloodPressure	BMI	Outcome
6	148	72	33.6	1
1	85	66	26.6	0
8	183	64	23.3	1
1	89	66	28.1	0

Here,

Outcome

- 0 → No Diabetes
- 1 → Diabetes

Step 3: Check Dataset Information

print(df.info())

print(df.describe())

print(df.isnull().sum())

This helps us understand

- Data types
- Missing values
- Statistical summary

Step 4: Separate Features and Target

X = df.drop("Outcome", axis=1)

y = df["Outcome"]

Input Features (X)

- Pregnancies
- Glucose
- BloodPressure
- BMI
- etc.

Target (y)

- Outcome

Step 5: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.2,
  random_state=42
)
```

Explanation

- **80%** Training Data
- **20%** Testing Data

Step 6: Create Logistic Regression Model

```
model = LogisticRegression()
```

Step 7: Train the Model

```
model.fit(X_train, y_train)
```

The model learns the relationship between

- Patient Health Data

↓

- Diabetes

Step 8: Predict Test Data

```
prediction = model.predict(X_test)
```

Example Output

```
array([1,0,1,0,0,1,1])
```

Step 9: Compare Actual vs Predicted

```
result = pd.DataFrame({
   "Actual": y_test,
   "Predicted": prediction
})

print(result.head(10))
```

Example Output

Actual	Predicted
1	1
0	0
1	1
0	0
1	0

Step 10: Calculate Accuracy

```
accuracy = accuracy_score(y_test, prediction)

print("Accuracy :", accuracy)
```

Example Output

```
Accuracy : 0.94
```

Meaning

The model correctly classified **94%** of patients.

Step 11: Confusion Matrix

```
cm = confusion_matrix(y_test, prediction)

print(cm)
```

Example Output

```
[[50  3]
[ 6 41]]
```

Meaning

	Predicted No	**Predicted Yes**
Actual No	50	3
Actual Yes	6	41

Step 12: Classification Report

```
print(classification_report(y_test, prediction))
```

Example Output

	precision	recall	f1-score
0	0.89	0.94	0.91
1	0.93	0.87	0.90
accuracy			0.91

The report provides:

- Precision
- Recall
- F1-Score
- Accuracy

Step 13: Predict New Patient

new_patient = [[2,120,70,30,0,25.5,0.45,28]]

prediction = model.predict(new_patient)

print(prediction)

Output

[0]

Meaning

Patient is predicted to have **No Diabetes**.

Step 14: Probability Prediction

Logistic Regression can predict probabilities.

probability = model.predict_proba(new_patient)

print(probability)

Example Output

[[0.82 0.18]]

Meaning

Probability of

No Diabetes = **82%**

Diabetes = **18%**

Practical Flow

Load Dataset

Preprocessing

Train-Test Split

Create Logistic Regression Model

Train Model

Prediction

Evaluate Accuracy

Confusion Matrix

Classification Report

Complete Program

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load Dataset
df = pd.read_csv("diabetes.csv")

# Features
X = df.drop("Outcome", axis=1)

# Target
y = df["Outcome"]

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.2,
  random_state=42
)

# Create Model
model = LogisticRegression()

# Train Model
model.fit(X_train, y_train)

# Prediction
```

```
prediction = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, prediction))

# Confusion Matrix
print(confusion_matrix(y_test, prediction))

# Classification Report
print(classification_report(y_test, prediction))
```

Expected Output

Accuracy : 0.94

Confusion Matrix

[[50 3]
[6 41]]

Precision : 0.91

Recall : 0.90

F1 Score : 0.90

(The exact values may differ depending on the dataset and the train-test split.)

Applications of Logistic Regression

- Disease Prediction
- Email Spam Detection
- Loan Approval
- Credit Card Fraud Detection
- Customer Churn Prediction
- Employee Attrition Prediction
- Cancer Detection
- Marketing Response Prediction

Observation

- Logistic Regression is suitable for **Binary Classification** problems.
- The algorithm predicts **class labels** (0 or 1) and can also estimate the **probability** of each class.
- Model performance is commonly evaluated using **Accuracy**, **Confusion Matrix**, **Precision**, **Recall**, and **F1-Score**.
- Logistic Regression is simple, fast, and works well when the classes are approximately linearly separable.

Practical: Logistic Regression (Binary Classification using Multiple Input Features)

Aim

To implement **Logistic Regression** for a **Binary Classification** problem using **multiple input features** and evaluate the model's performance.

Problem Statement

A hospital wants to predict whether a patient has **Diabetes**.

The prediction depends on **multiple health parameters**, not just one.

Input Features

- Pregnancies
- Glucose
- Blood Pressure
- Skin Thickness
- Insulin
- BMI
- Diabetes Pedigree Function
- Age

Output

- **0 → No Diabetes**
- **1 → Diabetes**

Since the output has only **two classes**, it is a **Binary Classification** problem.

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Step 2: Load Dataset

```
df = pd.read_csv("diabetes.csv")

print(df.head())
```

Example Dataset

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome

6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0

Step 3: Check Dataset

```
print(df.shape)

print(df.info())

print(df.isnull().sum())
```

Output

```
(768, 9)
```

No Missing Values

Step 4: Separate Input Features and Target

```
# Multiple Input Features

X = df[['Pregnancies',
       'Glucose',
       'BloodPressure',
       'SkinThickness',
       'Insulin',
       'BMI',
       'DiabetesPedigreeFunction',
       'Age']]

# Target Variable
```

y = df['Outcome']

Here,

Independent Variables (X)

- Pregnancies
- Glucose
- Blood Pressure
- Skin Thickness
- Insulin
- BMI
- Diabetes Pedigree Function
- Age

Dependent Variable (Y)

- Outcome

Step 5: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.20,
  random_state=42
)
```

Explanation

- 80% → Training Data
- 20% → Testing Data

Step 6: Create Logistic Regression Model

```
model = LogisticRegression(max_iter=1000)
```

Why use **max_iter=1000**?

Sometimes Logistic Regression needs more iterations to converge when multiple features are used.

Step 7: Train the Model

model.fit(X_train, y_train)

The model learns the relationship between

Pregnancies
Glucose
Blood Pressure
BMI
Age
↓
Diabetes Prediction

Step 8: Predict Test Data

y_pred = model.predict(X_test)

Example Output

array([0,1,1,0,0,1,0,1])

Step 9: Display Actual vs Predicted Values

result = pd.DataFrame({

"Actual": y_test.values,

"Predicted": y_pred

})

print(result.head(10))

Example Output

Actual	Predicted
1	1
0	0
0	0
1	0
1	1

Step 10: Calculate Accuracy

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy :", accuracy)

Example Output

Accuracy : 0.93

Meaning

The model correctly predicts **93%** of patients.

Step 11: Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)

Example Output

[[87 12]

[9 46]]

Meaning

	Predicted No	Predicted Yes
Actual No	87	12
Actual Yes	9	46

Step 12: Classification Report

print(classification_report(y_test, y_pred))

Example Output

	precision	recall	f1-score
0	0.91	0.88	0.89
1	0.87	0.91	0.89
accuracy			0.89

Step 13: Predict New Patient

Suppose

Feature	Value
Pregnancies	2
Glucose	135
Blood Pressure	75
Skin Thickness	28
Insulin	100
BMI	31.2
Pedigree	0.45

Age	40

```
new_patient = [[2,135,75,28,100,31.2,0.45,40]]

prediction = model.predict(new_patient)

print(prediction)
```

Output

```
[1]
```

Meaning

Patient is predicted to have **Diabetes**.

Step 14: Probability Prediction

```
probability = model.predict_proba(new_patient)

print(probability)
```

Example Output

```
[[0.18 0.82]]
```

Meaning

Class	Probability
No Diabetes	18%
Diabetes	82%

Step 15: Display Model Coefficients

```
print("Coefficients")

print(model.coef_)
```

print("Intercept")

print(model.intercept_)

Example Output

[[0.14

0.05

0.01

0.02

0.003

0.12

0.80

0.04]]

[-6.52]

These coefficients indicate how strongly each feature influences the prediction.

Complete Program

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load Dataset
df = pd.read_csv("diabetes.csv")

# Multiple Features
X = df[['Pregnancies',
```

```
        'Glucose',
        'BloodPressure',
        'SkinThickness',
        'Insulin',
        'BMI',
        'DiabetesPedigreeFunction',
        'Age']]

# Target
y = df['Outcome']

# Split
X_train,X_test,y_train,y_test=train_test_split(
   X,
   y,
   test_size=0.20,
   random_state=42
)

# Create Model
model = LogisticRegression(max_iter=1000)

# Train
model.fit(X_train,y_train)

# Prediction
y_pred=model.predict(X_test)

# Accuracy
print("Accuracy :",accuracy_score(y_test,y_pred))

# Confusion Matrix
print(confusion_matrix(y_test,y_pred))

# Classification Report
print(classification_report(y_test,y_pred))
```

Expected Output

Accuracy : 0.93

Confusion Matrix

[[87 12]

[9 46]]

Precision : 0.89

Recall : 0.90

F1 Score : 0.89

(Actual results may vary depending on the dataset and train-test split.)

Practical Workflow

Load Dataset

Select Multiple Features (X)

Select Target (Y)

Train-Test Split

Create Logistic Regression Model

Train Model

Predict Test Data

Evaluate Model

Predict New Patient

Observation

- Logistic Regression is a **Supervised Learning** algorithm used for **Binary Classification**.
- The model can learn from **multiple input features** simultaneously.
- It predicts both the **class label (0 or 1)** and the **probability** of each class.
- The model's performance is evaluated using **Accuracy**, **Confusion Matrix**, **Precision**, **Recall**, and **F1-Score**.
- Increasing the number of relevant features often improves prediction accuracy, provided the features are informative and not highly noisy.

Practical: Logistic Regression (Binary Classification using Polynomial Input Features)

Aim

To implement **Logistic Regression** for a **Binary Classification** problem using **Polynomial Features** to improve classification when the data is **non-linearly separable**.

Objective

- Load the dataset.
- Create Polynomial Features.
- Train a Logistic Regression model.
- Predict the output.
- Evaluate model performance.

Problem Statement

Suppose we want to predict whether a student will **Pass** or **Fail**.

The relationship between input and output is **non-linear**, so a simple Logistic Regression model may not perform well.

To solve this problem, we first convert the input features into **Polynomial Features**, then train a Logistic Regression model.

Dataset

Study Hours	Attendance	Pass
2	60	0
3	65	0
4	70	1
5	75	1
6	80	1
7	90	1
1	50	0
3	55	0
5	85	1
6	88	1

Where

Input Features (X)

- Study Hours

- Attendance

Target (Y)

- 0 → Fail
- 1 → Pass

Step 1: Import Libraries

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Step 2: Create Sample Dataset

```
data = {

  "StudyHours":[2,3,4,5,6,7,1,3,5,6],

  "Attendance":[60,65,70,75,80,90,50,55,85,88],

  "Pass":[0,0,1,1,1,1,0,0,1,1]

}

df = pd.DataFrame(data)

print(df)
```

Step 3: Select Features and Target

X = df[['StudyHours','Attendance']]

y = df['Pass']

Step 4: Create Polynomial Features

poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)

Polynomial Features Generated

Suppose

StudyHours = X1

Attendance = X2

New Features become

1

X1

X2

$X1^2$

X1X2

$X2^2$

Example

Original Input

StudyHours = 4

Attendance = 70

Polynomial Input

1

4

70

16

280

4900

These additional features help Logistic Regression learn **non-linear decision boundaries**.

Step 5: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_poly,
    y,
    test_size=0.20,
    random_state=42
)
```

Step 6: Create Logistic Regression Model

```
model = LogisticRegression(max_iter=1000)
```

Step 7: Train the Model

```
model.fit(X_train, y_train)
```

Step 8: Predict Test Data

prediction = model.predict(X_test)

Example Output

array([1,0,1])

Step 9: Calculate Accuracy

accuracy = accuracy_score(y_test, prediction)

print("Accuracy :", accuracy)

Example Output

Accuracy : 1.0

Step 10: Confusion Matrix

cm = confusion_matrix(y_test, prediction)

print(cm)

Example Output

[[2 0]

[0 3]]

Meaning

- True Positive = 3
- True Negative = 2
- False Positive = 0
- False Negative = 0

Step 11: Classification Report

print(classification_report(y_test, prediction))

Example Output

	precision	recall	f1-score
0	1.00	1.00	1.00
1	1.00	1.00	1.00
accuracy			1.00

Step 12: Predict New Student

Suppose

- Study Hours = 5
- Attendance = 78

Convert to Polynomial Features

new_student = [[5,78]]

new_student_poly = poly.transform(new_student)

prediction = model.predict(new_student_poly)

print(prediction)

Output

[1]

Meaning

Student is predicted to **Pass**.

Step 13: Predict Probability

probability = model.predict_proba(new_student_poly)

print(probability)

Example Output

[[0.08 0.92]]

Meaning

Class	Probability
Fail	8%
Pass	92%

Complete Program

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Dataset
data={

"StudyHours":[2,3,4,5,6,7,1,3,5,6],

"Attendance":[60,65,70,75,80,90,50,55,85,88],

"Pass":[0,0,1,1,1,1,0,0,1,1]

}
```

```
df=pd.DataFrame(data)

# Features
X=df[['StudyHours','Attendance']]

# Target
y=df['Pass']

# Polynomial Features
poly=PolynomialFeatures(degree=2)

X_poly=poly.fit_transform(X)

# Split Dataset
X_train,X_test,y_train,y_test=train_test_split(
X_poly,
y,
test_size=0.20,
random_state=42
)

# Create Model
model=LogisticRegression(max_iter=1000)

# Train Model
model.fit(X_train,y_train)

# Prediction
prediction=model.predict(X_test)

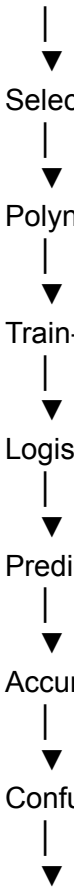
# Accuracy
print("Accuracy :",accuracy_score(y_test,prediction))

# Confusion Matrix
print(confusion_matrix(y_test,prediction))

# Classification Report
print(classification_report(y_test,prediction))
```

Workflow

Dataset



Why Use Polynomial Features?

Simple Logistic Regression creates a **linear decision boundary**.

Example

○ ○ ○ ○

● ● ● ●

However, many real-world datasets are **not linearly separable**.

Example

○ ○ ○

○ ○

-
-

Polynomial Features transform the original inputs into higher-dimensional features (such as **X^2**, **XY**, **Y^2**), allowing Logistic Regression to learn a **curved (non-linear) decision boundary** while still using the same Logistic Regression algorithm.

Applications

- Student Pass/Fail Prediction
- Disease Detection
- Fraud Detection
- Credit Card Approval
- Loan Approval
- Email Spam Detection
- Customer Churn Prediction

Observation

- Polynomial Features enable Logistic Regression to classify data that is not linearly separable.
- Increasing the polynomial degree creates more complex features, which can improve performance on non-linear datasets.
- Very high polynomial degrees may lead to **overfitting**, so the degree should be chosen carefully.
- After feature transformation, the training and evaluation steps remain the same as standard Logistic Regression.

Practical: Logistic Regression (Multiclass Classification) using Python (Scikit-Learn)

Aim

To implement **Logistic Regression** for a **Multiclass Classification** problem using the **Iris Flower Dataset** and evaluate the model's performance.

Objective

- Load the Iris dataset.
- Split the dataset into training and testing sets.
- Train a Logistic Regression model.
- Predict flower species.
- Evaluate the model using Accuracy, Confusion Matrix, and Classification Report.

Problem Statement

A botanist wants to identify the species of an Iris flower based on its physical characteristics.

Input Features

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Output Classes

- **0 → Setosa**
- **1 → Versicolor**
- **2 → Virginica**

Since there are **three output classes**, this is a **Multiclass Classification** problem.

About the Iris Dataset

The Iris dataset contains **150 flower samples**.

There are **4 input features** and **3 output classes**.

Feature	Description
Sepal Length	Length of sepal (cm)
Sepal Width	Width of sepal (cm)
Petal Length	Length of petal (cm)
Petal Width	Width of petal (cm)

Target Classes

Value	Flower Name
0	Setosa
1	Versicolor
2	Virginica

Step 1: Import Required Libraries

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
```

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Step 2: Load Dataset

```
iris = load_iris()

X = iris.data

y = iris.target
```

Step 3: Display Dataset Information

```
print("Features")

print(iris.feature_names)

print()

print("Target Classes")

print(iris.target_names)
```

Output

```
Features

['sepal length (cm)',
'sepal width (cm)',
'petal length (cm)',
'petal width (cm)']

Target Classes

['setosa'
'versicolor'
'virginica']
```

Step 4: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.20,
  random_state=42
)
```

Training Data = 80%

Testing Data = 20%

Step 5: Create Logistic Regression Model

```
model = LogisticRegression(max_iter=1000)
```

Step 6: Train the Model

```
model.fit(X_train, y_train)
```

The model learns the relationship between

Sepal Length

Sepal Width

Petal Length

Petal Width

↓

Flower Species

Step 7: Predict Test Data

prediction = model.predict(X_test)

Example Output

array([1,0,2,1,1,0,2])

Step 8: Compare Actual vs Predicted

result = pd.DataFrame({

  "Actual": y_test,

  "Predicted": prediction

})

print(result.head(10))

Example Output

Actual	Predicted
1	1
0	0
2	2
1	1
0	0

Step 9: Calculate Accuracy

accuracy = accuracy_score(y_test, prediction)

print("Accuracy :", accuracy)

Example Output

Accuracy : 1.0

Meaning

The model correctly classified **100%** of the test samples.

Step 10: Confusion Matrix

cm = confusion_matrix(y_test, prediction)

print(cm)

Example Output

[[10 0 0]

[0 9 0]

[0 0 11]]

Interpretation

Actual \ Predicted	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	0
Virginica	0	0	11

Step 11: Classification Report

print(classification_report(

```
    y_test,
    prediction,
    target_names=iris.target_names
))
```

Example Output

```
              precision    recall  f1-score

setosa           1.00      1.00      1.00

versicolor       1.00      1.00      1.00

virginica        1.00      1.00      1.00

accuracy                             1.00
```

Step 12: Predict a New Flower

Suppose a flower has

Feature	Value
Sepal Length	5.8
Sepal Width	3.0
Petal Length	5.1
Petal Width	1.8

```
new_flower = [[5.8,3.0,5.1,1.8]]

prediction = model.predict(new_flower)

print(prediction)
```

Output

```
[2]
```

Meaning

Virginica

Step 13: Display Flower Name

flower = iris.target_names[prediction[0]]

print(flower)

Output

virginica

Step 14: Predict Probability

probability = model.predict_proba(new_flower)

print(probability)

Example Output

[[0.01 0.05 0.94]]

Meaning

Flower	Probability
Setosa	1%
Versicolor	5%
Virginica	94%

The model predicts **Virginica** because it has the highest probability.

Complete Program

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load Dataset
iris = load_iris()

X = iris.data

y = iris.target

# Split Dataset
X_train,X_test,y_train,y_test=train_test_split(
X,
y,
test_size=0.20,
random_state=42
)

# Create Model
model=LogisticRegression(max_iter=1000)

# Train Model
model.fit(X_train,y_train)

# Prediction
prediction=model.predict(X_test)

# Accuracy
print("Accuracy :",accuracy_score(y_test,prediction))

# Confusion Matrix
print(confusion_matrix(y_test,prediction))

# Classification Report
print(classification_report(
```

```
y_test,
prediction,
target_names=iris.target_names
))
```

Expected Output

Accuracy : 1.0

Confusion Matrix

[[10 0 0]

[0 9 0]

[0 0 11]]

Precision : 1.00

Recall : 1.00

F1 Score : 1.00

(Actual results may vary slightly depending on the train-test split.)

Practical Workflow

Load Iris Dataset



Select Features & Target

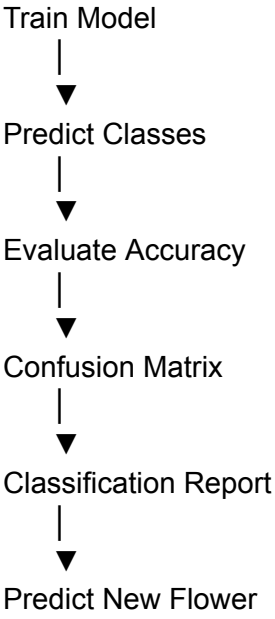


Train-Test Split



Create Logistic Regression Model





Observation

- Logistic Regression can perform **Multiclass Classification** in addition to Binary Classification.
- The **Iris dataset** contains **three flower species**, making it a standard example of multiclass classification.
- Scikit-Learn automatically uses a suitable multiclass strategy (such as **One-vs-Rest (OvR)** or the **multinomial** approach, depending on the solver and settings).
- The model predicts both the **class label** and the **probability** for each class.
- Performance is evaluated using **Accuracy**, **Confusion Matrix**, **Precision**, **Recall**, and **F1-Score**.

Tab 9

Confusion Matrix

Introduction

After training a **Classification** model, we need to evaluate how well it performs.

Accuracy alone is often not enough because it does not show **which predictions were correct or incorrect**.

To obtain a detailed evaluation of a classification model, we use the **Confusion Matrix**.

What is a Confusion Matrix?

A **Confusion Matrix** is a table used to evaluate the performance of a **classification model** by comparing the **actual class labels** with the **predicted class labels**.

It shows:

- Correct Predictions
- Incorrect Predictions
- Types of Errors made by the model

Definition

> A Confusion Matrix is a table that summarizes the performance of a classification model by comparing the actual values with the predicted values.

Why Do We Need a Confusion Matrix?

Suppose a disease prediction model gives **95% accuracy**.

Is the model really good?

Not necessarily.

It may predict healthy patients correctly but fail to identify sick patients.

A Confusion Matrix helps us understand:

- How many predictions are correct?
- How many predictions are incorrect?
- What type of mistakes the model makes?

Binary Classification Example

Suppose we want to predict whether a patient has diabetes.

Actual Classes

- Positive (Diabetes)
- Negative (No Diabetes)

Predicted Classes

- Positive
- Negative

The Confusion Matrix is

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Understanding the Four Terms

1. True Positive (TP)

The model correctly predicts a **Positive** class.

Example

Actual

Diabetes

Predicted

Diabetes

Correct Prediction.

---

2. True Negative (TN)

The model correctly predicts a **Negative** class.

Example

Actual

No Diabetes

Predicted

No Diabetes

Correct Prediction.

---

3. False Positive (FP)

The model predicts **Positive**, but the actual class is **Negative**.

This is also called a **Type-I Error**.

Example

Actual

No Diabetes

Predicted

Diabetes

The patient is healthy, but the model incorrectly predicts disease.

---

4. False Negative (FN)

The model predicts **Negative**, but the actual class is **Positive**.

This is called a **Type-II Error**.

Example

Actual

Diabetes

Predicted

No Diabetes

This is dangerous because the patient actually has the disease.

Visual Representation

	Predicted Class	
	Positive	Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Example

Suppose we test **100 patients**.

Actual	**Predicted**

Diabetes	Diabetes
Diabetes	Diabetes
Diabetes	No Diabetes
No Diabetes	Diabetes
No Diabetes	No Diabetes

After testing,

	Predicted Positive	**Predicted Negative**
Actual Positive	45	5
Actual Negative	10	40

Therefore,

TP = 45

TN = 40

FP = 10

FN = 5

Interpretation

True Positive

45 patients correctly predicted as diabetic.

True Negative

40 healthy patients correctly identified.

False Positive

10 healthy patients wrongly predicted as diabetic.

False Negative

5 diabetic patients wrongly predicted as healthy.

Evaluation Metrics from Confusion Matrix

The Confusion Matrix is used to calculate several important evaluation metrics.

---

1. Accuracy

1. Accuracy

Accuracy measures the overall correctness of the model.

Formula

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Example

TP = 45

TN = 40

FP = 10

FN = 5

$$Accuracy = \frac{45 + 40}{45 + 40 + 10 + 5}$$

$$Accuracy = \frac{85}{100} = 0.85$$

Accuracy = **85%**

2. Precision

2. Precision

Precision tells us:

Out of all predicted Positive cases, how many were actually Positive?

Formula

$$Precision = \frac{TP}{TP + FP}$$

Example

$$Precision = \frac{45}{45 + 10}$$

$$Precision = 0.818$$

Precision = **81.8%**

3. Recall (Sensitivity)

3. Recall (Sensitivity)

Recall tells us:

Out of all actual Positive cases, how many were correctly identified?

Formula

$$Recall = \frac{TP}{TP + FN}$$

Example

$$Recall = \frac{45}{45 + 5}$$

$$Recall = 0.90$$

Recall = **90%**

4. F1 Score

F1 Score is the harmonic mean of Precision and Recall.

Formula

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Example

Precision = 0.818

Recall = 0.90

$$F1 = 0.857$$

F1 Score = **85.7%**

Complete Example

Suppose

```
TP = 70

TN = 20

FP = 5

FN = 5
```

Accuracy

$$Accuracy = \frac{70 + 20}{100} = 90\%$$

Precision

$$Precision = \frac{70}{75} = 93.3\%$$

Recall

$$Recall = \frac{70}{75} = 93.3\%$$

F1 Score

Approximately **93.3%**

Confusion Matrix for Multi-Class Classification

For three classes

- Cat
- Dog
- Horse

The matrix becomes

Actual \ Predicted	Cat	Dog	Horse
Cat	20	1	0
Dog	2	18	1
Horse	0	2	16

The diagonal values represent **correct predictions**, while the off-diagonal values represent **misclassifications**.

Python Example

```
from sklearn.metrics import confusion_matrix

# Actual Values
y_true = [1,0,1,1,0,0,1,0]

# Predicted Values
y_pred = [1,0,1,0,0,1,1,0]

cm = confusion_matrix(y_true,y_pred)

print(cm)
```

Output

[[3 1]

[1 3]]

Meaning

	Predicted 0	Predicted 1
Actual 0	3	1
Actual 1	1	3

Confusion Matrix using Seaborn

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm,
        annot=True,
        cmap="Blues",
        fmt="d")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()
```

Applications

- Disease Diagnosis
- Email Spam Detection
- Face Recognition
- Fraud Detection

- Customer Churn Prediction
- Loan Approval
- Sentiment Analysis

Advantages

- Gives detailed model evaluation.
- Shows the types of prediction errors.
- Used to calculate Accuracy, Precision, Recall, and F1 Score.
- Suitable for both Binary and Multiclass Classification.
- Helps compare different classification models.

Limitations

- Can be difficult to interpret for datasets with many classes.
- Accuracy derived from the confusion matrix may be misleading for highly imbalanced datasets.
- Should be used together with metrics such as Precision, Recall, and F1 Score for a complete evaluation.

Key Points to Remember

- A **Confusion Matrix** is used to evaluate **Classification** models.
- It compares the **Actual** class labels with the **predicted** class labels.
- It consists of **True Positive (TP)**, **True Negative (TN)**, **False Positive (FP)**, and **False Negative (FN)**.
- Important evaluation metrics such as **Accuracy**, **Precision**, **Recall**, and **F1 Score** are calculated from the confusion matrix.
- In **Binary Classification**, the confusion matrix is a **2 × 2** table, while in **Multiclass Classification**, it becomes an **N × N** matrix, where **N** is the number of classes.

Confusion Matrix, Sensitivity, Precision, Recall & F1-Score

Introduction

After training a **Classification Model**, we need to evaluate how well it performs.

A model may have high accuracy but still make critical mistakes, especially in applications such as disease detection or fraud detection.

To evaluate a classification model in detail, we use the **Confusion Matrix** and its associated performance metrics:

- Accuracy
- Sensitivity (Recall)
- Specificity
- Precision
- F1-Score

What is a Confusion Matrix?

A **Confusion Matrix** is a table that compares the **Actual Values** with the **Predicted Values** of a classification model.

It helps us understand:

- Correct Predictions
- Wrong Predictions
- Type of Errors

Binary Confusion Matrix

Actual / Predicted	Positive	Negative

Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

Meaning of Each Term

1. True Positive (TP)

The model correctly predicts the **Positive** class.

Example

Actual

Patient has Diabetes

Predicted

Patient has Diabetes

Correct Prediction

---

2. True Negative (TN)

The model correctly predicts the **Negative** class.

Example

Actual

Patient is Healthy

Predicted

Patient is Healthy

Correct Prediction

---

3. False Positive (FP)

The model predicts **Positive**, but the actual class is **Negative**.

Also called **Type I Error**.

Example

Actual

Healthy

Predicted

Diabetes

Wrong Prediction

---

4. False Negative (FN)

The model predicts **Negative**, but the actual class is **Positive**.

Also called **Type-II Error**.

Example

Actual

Diabetes

Predicted

Healthy

Wrong Prediction

This is dangerous in medical diagnosis because the patient actually has the disease.

Example Confusion Matrix

Suppose a disease prediction model tested **100 patients**.

Actual / Predicted	Positive	Negative
Positive	45	5
Negative	10	40

Therefore,

TP = 45

FN = 5

FP = 10

TN = 40

---

1. Accuracy

Definition

Accuracy measures the percentage of total correct predictions.

Formula

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Example

TP = 45

TN = 40

FP = 10

FN = 5

$$Accuracy = \frac{45 + 40}{45 + 40 + 10 + 5}$$

$$Accuracy = \frac{85}{100}$$

Accuracy = **85%**

2. Sensitivity (Recall or True Positive Rate)

Definition

Sensitivity measures the ability of the model to correctly identify **actual positive cases**.

It answers the question:

> Out of all patients who actually have the disease, how many were correctly identified?

Sensitivity is also called the **Recall** or **True Positive Rate (TPR)**.

Formula

$$Sensitivity = \frac{TP}{TP + FN}$$

Example

TP = 45

FN = 5

$$Sensitivity = \frac{45}{45 + 5}$$

$$Sensitivity = \frac{45}{50}$$

Sensitivity = **90%**

Interpretation

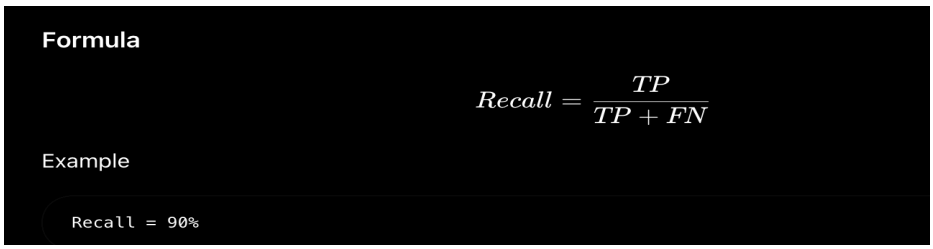
Out of 50 diabetic patients,

The model correctly identified **45**.

---

3. Recall

Recall is **the same as Sensitivity**.



Why Two Names?

Different fields use different terminology.

Medical Field	Machine Learning
Sensitivity	Recall

Both represent the same metric.

4. Precision

Definition

Precision measures how many of the predicted positive cases are actually positive.

It answers:

> Out of all patients predicted as diabetic, how many actually have diabetes?

Formula

$$Precision = \frac{TP}{TP + FP}$$

Example

TP = 45

FP = 10

$$Precision = \frac{45}{45 + 10}$$

$$Precision = \frac{45}{55}$$

Precision = **81.8%**

Precision vs Recall

Suppose

100 patients

Actually diabetic = 20

Model predicts

Positive = 50

Only 20 are actually diabetic.

Precision becomes low.

However,

if all diabetic patients are detected,

Recall becomes high.

---

5. Specificity

Definition

Specificity measures the ability to correctly identify **negative cases**.

5. Specificity

Definition

Specificity measures the ability to correctly identify **negative cases**.

Formula

$$Specificity = \frac{TN}{TN + FP}$$

Example

TN = 40

FP = 10

$$Specificity = \frac{40}{50}$$

Specificity = **80%**

6. F1 Score

Definition

The F1 Score is the **Harmonic Mean of Precision and Recall**.

It balances both Precision and Recall into a single metric.

6. F1 Score

Definition

The F1 Score is the **Harmonic Mean of Precision and Recall**.

It balances both Precision and Recall into a single metric.

Formula

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Example

Precision = 81.8%

Recall = 90%

$$F1 = 2 \times \frac{0.818 \times 0.90}{0.818 + 0.90}$$

$$F1 = 0.857$$

F1 Score = **85.7%**

Why Harmonic Mean?

Suppose

Precision = 100%

Recall = 0%

Arithmetic Mean

50%

This is misleading.

The Harmonic Mean gives

0%

which correctly reflects poor overall performance.

Complete Example

Suppose

TP = 70

TN = 20

FP = 5

FN = 5

Accuracy

$$Accuracy = \frac{70 + 20}{100} = 90\%$$

Sensitivity

$$Sensitivity = \frac{70}{75} = 93.3\%$$

Precision

$$Precision = \frac{70}{75} = 93.3\%$$

Specificity

$$Specificity = \frac{20}{25} = 80\%$$

F1 Score

$$F1 = 93.3\%$$

Comparison Table

Metric	Formula	Purpose
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Overall Correct Predictions
Sensitivity	TP/(TP+FN)	Detect Positive Cases
Recall	TP/(TP+FN)	Same as Sensitivity
Precision	TP/(TP+FP)	Correct Positive Predictions
Specificity	TN/(TN+FP)	Detect Negative Cases
F1 Score	2×(Precision×Recall)/(Precision+Recall)	Balance Precision & Recall

When to Use Each Metric

Situation	Best Metric
Balanced Dataset	Accuracy
Disease Detection	Sensitivity (Recall)
Spam Detection	Precision
Fraud Detection	Recall
Imbalanced Dataset	F1 Score
Medical Diagnosis	Sensitivity + Specificity

Python Example

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

y_true = [1,0,1,1,0,0,1,0]

y_pred = [1,0,1,0,0,1,1,0]

print("Accuracy :",accuracy_score(y_true,y_pred))

print("Precision :",precision_score(y_true,y_pred))

print("Recall :",recall_score(y_true,y_pred))

print("F1 Score :",f1_score(y_true,y_pred))
```

```
print(confusion_matrix(y_true,y_pred))
```

Output

Accuracy : 0.75

Precision : 0.75

Recall : 0.75

F1 Score : 0.75

[[3 1]

[1 3]]

Applications

- Disease Diagnosis
- Fraud Detection
- Face Recognition
- Loan Approval
- Email Spam Detection
- Customer Churn Prediction
- Fake News Detection
- Sentiment Analysis

Key Points to Remember

- A **Confusion Matrix** evaluates the performance of a **classification model** by comparing actual and predicted class labels.
- It consists of **True Positive (TP)**, **True Negative (TN)**, **False Positive (FP)**, and **False Negative (FN)**.
- **Accuracy** measures the overall correctness of the model.
- **Sensitivity** and **Recall** are the **same metric**, measuring the proportion of actual positive cases correctly identified.

- **Precision** measures how many predicted positive cases are truly positive.
- **Specificity** measures how many actual negative cases are correctly identified.
- **F1 Score** combines Precision and Recall into a single metric using the harmonic mean.
- For **imbalanced datasets**, **Precision**, **Recall**, and **F1 Score** are generally more informative than Accuracy alone.

Tab 10

Imbalanced Datasets in Machine Learning

Introduction

In Machine Learning, the performance of a classification model depends greatly on the distribution of the data.

If one class contains significantly more samples than the other class, the dataset is called an **Imbalanced Dataset**.

Imbalanced datasets are common in real-world applications such as **fraud detection**, **disease diagnosis**, and **network intrusion detection**.

What is an Imbalanced Dataset?

An **Imbalanced Dataset** is a dataset in which the number of samples in one class is much larger than the number of samples in the other class(es).

Definition

> An Imbalanced Dataset is a dataset where one class (majority class) has significantly more observations than another class (minority class).

Balanced Dataset vs Imbalanced Dataset

Balanced Dataset

A balanced dataset contains approximately the **same number of samples** in each class.

Example

Student	Result
1	Pass
2	Fail
3	Pass
4	Fail
5	Pass
6	Fail

Here,

Pass = 3

Fail = 3

This is a **Balanced Dataset**.

Imbalanced Dataset

An imbalanced dataset has **unequal class distribution**.

Example

Student	Result
1	Pass
2	Pass
3	Pass
4	Pass
5	Pass
6	Pass

7	Pass
8	Pass
9	Pass
10	Fail

Here,

Pass = 9

Fail = 1

This is an **Imbalanced Dataset**.

Visual Representation

Balanced Dataset

Pass : ▉▉▉▉▉▉▉▉▉

Fail : ▉▉▉▉▉▉▉▉▉

Imbalanced Dataset

Pass : ▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉

Fail : ▉▉

Majority Class and Minority Class

Suppose

Healthy Patients = 950

Diabetic Patients = 50

Then

- **Majority Class → Healthy**
- **Minority Class → Diabetes**

Why is an Imbalanced Dataset a Problem?

Suppose we have **1000 patients**.

Healthy = 980

Diabetes = 20

A poor model predicts

Healthy

Healthy

Healthy

Healthy

Healthy

...

Healthy

for every patient.

Accuracy Calculation

Correct Predictions

980

Wrong Predictions

20

Accuracy

Accuracy=9801000Accuracy=\frac{980}{1000}Accuracy=1000980

Accuracy = **98%**

This looks excellent.

However,

The model **failed to detect every diabetic patient**.

Therefore,

High Accuracy does not always indicate a good model.

Real-Life Example

Suppose an airport security system screens **10,000 passengers**.

Only

10

Passengers

Carry Illegal Items

The remaining

9990

Passengers

Are Safe

If the model predicts

Safe

Safe

Safe

Safe

for everyone,

Accuracy

99.9%

But

The model fails to detect **all criminals**.

This model is useless despite its high accuracy.

Common Examples of Imbalanced Datasets

Application	Majority Class	Minority Class
Disease Detection	Healthy	Disease
Fraud Detection	Genuine Transaction	Fraud Transaction
Email Filtering	Normal Email	Spam Email
Credit Card Fraud	Valid Payment	Fraud Payment
Intrusion Detection	Normal Traffic	Attack
Manufacturing	Good Product	Defective Product

Why Does Imbalance Occur?

Some events are naturally rare.

Examples

- Cancer Patients

- Fraud Transactions
- Terrorist Activities
- Earthquakes
- Machine Failures

These rare events create an imbalanced dataset.

Problems Caused by Imbalanced Datasets

- High Accuracy but Poor Performance
- The model becomes biased toward the majority class.
- Minority class predictions become poor.
- Recall decreases.
- Important cases may be missed.

Example

Suppose

100 Patients

95 Healthy

5 Diabetes

Prediction

Healthy = 100

Confusion Matrix

Actual / Predicted	Healthy	Diabetes
Healthy	95	0
Diabetes	5	0

Accuracy

95%95\%95%

Looks good.

Recall

$$Recall = \frac{TP}{TP + FN}$$

TP = 0

FN = 5

Recall

0%

The model cannot identify any diabetic patients.

Which Metrics Should We Use?

For imbalanced datasets,

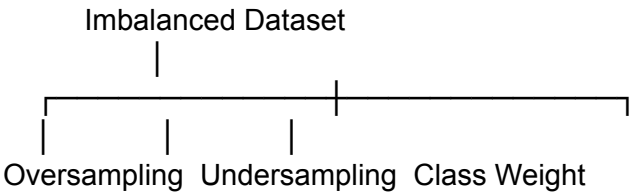
Accuracy is NOT sufficient.

Instead, use:

- Precision
- Recall (Sensitivity)
- F1 Score
- ROC-AUC Score
- Precision-Recall Curve

How to Handle Imbalanced Datasets?

There are several techniques.



---

1. Oversampling

Oversampling increases the number of minority class samples.

Example

Before

Healthy = 900

Disease = 100

After Oversampling

Healthy = 900

Disease = 900

Popular Method

- **SMOTE (Synthetic Minority Over-sampling Technique)**

Advantages

- Improves Recall.

- Retains all majority class samples.
- Better model performance.

Disadvantages

- Larger dataset.
- Longer training time.
- Possible overfitting if simple duplication is used.

---

2. Undersampling

Undersampling reduces the number of majority class samples.

Before

Healthy = 900

Disease = 100

After

Healthy = 100

Disease = 100

Advantages

- Faster training.
- Smaller dataset.

Disadvantages

- Valuable data may be lost.
- Lower accuracy if too many majority samples are removed.

---

3. Class Weight

Instead of changing the dataset,

the algorithm assigns **higher importance** to minority class samples.

Example

Healthy

Weight = 1

Disease

Weight = 10

The model pays more attention to minority samples during training.

Python Example

Check Class Distribution

```
import pandas as pd

df = pd.read_csv("diabetes.csv")

print(df["Outcome"].value_counts())
```

Output

```
0   500

1   268
```

Dataset is imbalanced.

Logistic Regression with Class Weight

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
  class_weight="balanced"
)

model.fit(X_train,y_train)
```

The model automatically assigns higher weights to the minority class.

SMOTE Example

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)

X_resampled,y_resampled = smote.fit_resample(X,y)
```

Check Distribution

```
print(pd.Series(y_resampled).value_counts())
```

Output

0 500

1 500

Now the dataset is balanced.

Advantages of Handling Imbalanced Data

- Better Recall.
- Better Precision.
- Higher F1 Score.
- Improved minority class prediction.

- More reliable model.

Applications

- Cancer Detection
- Fraud Detection
- Spam Detection
- Credit Card Fraud
- Cyber Security
- Network Intrusion Detection
- Manufacturing Defect Detection

Summary

Balanced Dataset	Imbalanced Dataset
Equal number of samples	Unequal number of samples
Accuracy is reliable	Accuracy may be misleading
Easy to train	Difficult to train
Less biased	More biased toward majority class

Key Points to Remember

- An **Imbalanced Dataset** has **unequal class distribution**, where one class contains significantly more samples than another.
- The larger class is called the **Majority Class**, while the smaller class is called the **Minority Class**.
- High **Accuracy** can be misleading because a model may correctly predict the majority class while completely ignoring the minority class.
- For imbalanced datasets, **Precision**, **Recall (Sensitivity)**, **F1 Score**, and **ROC-AUC** are more informative evaluation metrics than Accuracy alone.
- Common techniques for handling imbalanced datasets include:

    - **Oversampling** (e.g., SMOTE)
    - **Undersampling**
    - **Class Weighting**
- Imbalanced datasets are frequently encountered in applications such as **fraud detection**, **medical diagnosis**, **spam filtering**, and **network intrusion detection**.

Tab 11

Naive Bayes Algorithm

Introduction

Naïve Bayes is one of the simplest and fastest **Supervised Machine Learning Classification** algorithms.

It is based on **Bayes' Theorem**, which calculates the probability of an event based on prior knowledge.

Naïve Bayes is widely used in:

- Email Spam Detection
- Sentiment Analysis
- Text Classification
- News Categorization
- Disease Prediction

What is Naïve Bayes?

Naïve Bayes is a **probabilistic classification algorithm** that predicts the class of a data point using **Bayes' Theorem**.

It assumes that all input features are **independent** of each other.

Definition

> Naïve Bayes is a supervised machine learning algorithm based on Bayes' Theorem that predicts the probability of a data point belonging to a particular class by assuming that all input features are independent.

Why is it called "Naive"?

The algorithm is called **Naïve** because it assumes that all input features are **independent** of each other.

Example

Suppose we predict whether a person buys a laptop based on

- Age
- Salary
- Education

Naïve Bayes assumes

Age ⊥ Salary

Salary ⊥ Education

Age ⊥ Education

In reality, these features may be related.

This assumption is called the **Naïve Assumption**.

Bayes' Theorem



Understanding the Formula

Suppose

- A = Patient has Diabetes
- B = Blood Sugar is High

Then

$$P(A|B)$$

means

Probability that the patient has diabetes given that the blood sugar level is high.

Components of Bayes' Theorem

1. Prior Probability

The probability before observing any evidence.

Example

Out of 100 people

20 have Diabetes

Therefore

Therefore

$$P(Diabetes) = \frac{20}{100} = 0.20$$

2. Likelihood

Probability of observing evidence if the event is true.

Example

Among diabetic patients,

18 have High Sugar

Therefore

Therefore

$$P(HighSugar|Diabetes) = \frac{18}{20} = 0.90$$

3. Evidence

Overall probability of observing the evidence.

Example

Out of 100 people,

30 have High Sugar

Therefore

Therefore

$$P(HighSugar) = \frac{30}{100} = 0.30$$

4. Posterior Probability

Probability after considering the evidence.

Example

Using Bayes' Theorem

Using Bayes' Theorem

$$P(Diabetes|HighSugar) = \frac{0.90 \times 0.20}{0.30} = 0.60$$

So,

There is a **60% probability** that a person has diabetes given that their blood sugar level is high.

Working of Naive Bayes

Suppose we want to classify an email.

Features

- Contains "Free"
- Contains "Lottery"
- Contains "Offer"

↓

Naïve Bayes calculates

Probability(Spam)

Probability(Not Spam)

Whichever probability is higher,

that class becomes the prediction.

Example

Suppose

P(Spam)=0.70

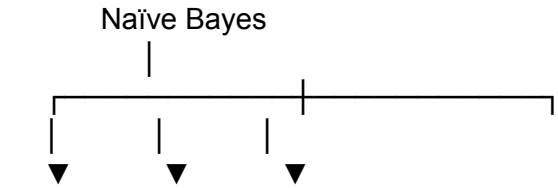
P(Not Spam)=0.30

Prediction

Spam

Types of Naïve Bayes

There are three major types.



---

1. Gaussian Naive Bayes

Used for **continuous numerical data**.

Examples

- Height
- Weight
- Temperature
- Salary

Assumes that the data follows a **Normal (Gaussian) Distribution**.

2. Multinomial Naive Bayes

Used for

- Text Classification
- Email Classification
- Word Frequency

Examples

- Number of times a word appears
- Document Classification

---

3. Bernoulli Naive Bayes

Used for **Binary Features**.

Example

Word Exists?

Yes

No

Suitable for

- Spam Detection
- Binary Text Classification

Example Dataset

Suppose we want to predict whether a student will pass.

Study Hours	Attendance	Result
2	60	Fail
4	75	Pass
5	80	Pass
1	50	Fail
6	90	Pass

Input

- Study Hours
- Attendance

Output

- Pass
- Fail

Naïve Bayes calculates

P(Pass)

P(Fail)

The larger probability becomes the prediction.

Steps of Naive Bayes Algorithm

1. Collect the dataset.
2. Calculate Prior Probability.
3. Calculate Likelihood.
4. Apply Bayes' Theorem.
5. Compute Posterior Probability.
6. Predict the class with the highest probability.

Advantages

- Very simple to implement.
- Fast training and prediction.
- Works well with high-dimensional data.
- Excellent for text classification.
- Requires relatively small training data.

Disadvantages

- Assumes feature independence.
- Performance decreases when features are highly correlated.
- Zero-frequency problem (handled using **Laplace Smoothing**).

Laplace Smoothing

Sometimes a feature never appears in the training data for a particular class.

This causes the probability to become **zero**.

Example

Suppose

P("Lottery"|Spam)=0

Then the final probability also becomes **0**, even if all other features strongly indicate spam.

To avoid this, **Laplace Smoothing** adds **1** to every feature count before calculating probabilities.

This ensures that no probability becomes exactly zero.

Applications

- Email Spam Detection
- Disease Prediction
- Sentiment Analysis
- Fake News Detection
- Document Classification
- Language Detection
- Product Recommendation
- News Categorization

Python Example

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score

# Load Dataset
df = pd.read_csv("diabetes.csv")

# Features
X = df.drop("Outcome", axis=1)

# Target
y = df["Outcome"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.20,
  random_state=42
)

# Create Model
model = GaussianNB()
```

```
# Train Model
model.fit(X_train, y_train)

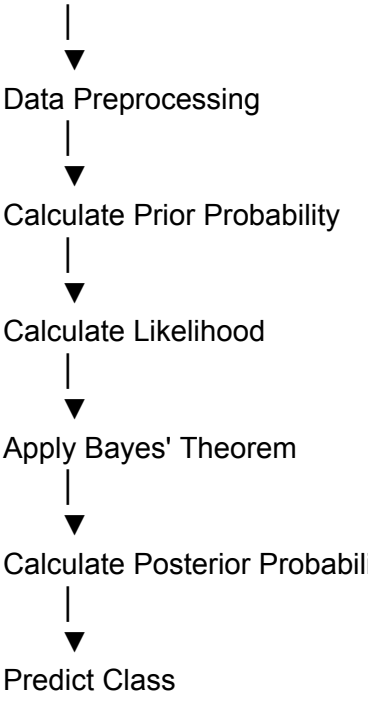
# Prediction
prediction = model.predict(X_test)

# Accuracy
print("Accuracy :", accuracy_score(y_test, prediction))
```

Sample Output

Accuracy: 0.88

Naive Bayes Workflow



Naive Bayes vs Logistic Regression

Feature	Naïve Bayes	Logistic Regression
Algorithm Type	Probabilistic	Linear Classification
Based On	Bayes' Theorem	Sigmoid Function
Output	Probability of each class	Probability of each class
Assumption	Features are independent	No independence assumption
Training Speed	Very Fast	Moderate
Best For	Text Classification	General Classification
Handles Correlated Features	Poorly	Better than Naïve Bayes

Real-Life Applications

Application	Example
Spam Detection	Spam / Not Spam
Disease Diagnosis	Positive / Negative
Sentiment Analysis	Positive / Neutral / Negative
News Classification	Sports, Politics, Business
Language Detection	English, Hindi, Bengali
Document Classification	Research Papers, Books, Articles

Key Points to Remember

- **Naive Bayes** is a **Supervised Machine Learning Classification** algorithm.
- It is based on **Bayes' Theorem**.

- It assumes that all input features are **independent** of one another (Naïve Assumption).
- It predicts the class with the **highest posterior probability**.
- It is fast, simple, and highly effective for **text classification** problems.
- The three main types are:
    - **Gaussian Naïve Bayes** (continuous numerical data)
    - **Multinomial Naïve Bayes** (word counts and text data)
    - **Bernoulli Naïve Bayes** (binary features)
- **Laplace Smoothing** is used to prevent zero probabilities during prediction.

Practical: Naive Bayes Classification using Python (Scikit-Learn)

Aim

To implement the **Naive Bayes Classification Algorithm** using Python and Scikit-Learn for predicting whether a patient has **Diabetes**.

Objective

- Load the dataset.
- Split the dataset into training and testing sets.
- Train the Naïve Bayes model.
- Predict the output.
- Evaluate the model using Accuracy, Confusion Matrix, Precision, Recall, and F1-Score.

Problem Statement

A hospital wants to predict whether a patient has **Diabetes** based on medical information.

Input Features

- Pregnancies
- Glucose
- Blood Pressure
- Skin Thickness
- Insulin
- BMI
- Diabetes Pedigree Function
- Age

Output

- **0 → No Diabetes**
- **1 → Diabetes**

Since the output has only **two classes**, this is a **Binary Classification** problem.

Step 1: Import Required Libraries

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Step 2: Load Dataset

```
df = pd.read_csv("diabetes.csv")

print(df.head())
```

Example Dataset

Pregnancies	Glucose	BloodPressure	BMI	Age	Outcome
6	148	72	33.6	50	1
1	85	66	26.6	31	0
8	183	64	23.3	32	1
1	89	66	28.1	21	0

Step 3: Check Dataset

```
print(df.shape)

print(df.info())

print(df.isnull().sum())
```

Output

(768, 9)

No Missing Values

Step 4: Separate Features and Target

```
X = df.drop("Outcome", axis=1)

y = df["Outcome"]
```

Independent Variables (X)

- Pregnancies
- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI
- DiabetesPedigreeFunction
- Age

Dependent Variable (Y)

- Outcome

Step 5: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
  X,
```

```
    y,
    test_size=0.20,
    random_state=42
)
```

Explanation

- Training Data = 80%
- Testing Data = 20%

Step 6: Create Naïve Bayes Model

model = GaussianNB()

Why GaussianNB?

Because the diabetes dataset contains **continuous numerical features**.

Step 7: Train the Model

model.fit(X_train, y_train)

The model learns

Pregnancies

Glucose

Blood Pressure

BMI

Age

↓

Diabetes Prediction

Step 8: Predict Test Data

```
prediction = model.predict(X_test)
```

Example Output

```
array([1,0,1,0,0,1,1])
```

Step 9: Compare Actual vs Predicted

```
result = pd.DataFrame({
   "Actual": y_test.values,
   "Predicted": prediction
})
print(result.head(10))
```

Example Output

Actual	Predicted
1	1
0	0
1	1
0	0
1	0

Step 10: Calculate Accuracy

```
accuracy = accuracy_score(y_test, prediction)
print("Accuracy :", accuracy)
```

Example Output

Accuracy : 0.88

Meaning

The model correctly classified **88%** of patients.

Step 11: Confusion Matrix

cm = confusion_matrix(y_test, prediction)

print(cm)

Example Output

[[84 15]

[11 44]]

Interpretation

Actual / Predicted	No Diabetes	Diabetes
No Diabetes	84	15
Diabetes	11	44

Step 12: Classification Report

print(classification_report(y_test, prediction))

Example Output

	precision	recall	f1-score
0	0.88	0.85	0.86
1	0.81	0.86	0.83

accuracy 0.84

The report provides

- Precision
- Recall
- F1-Score
- Accuracy

Step 13: Predict New Patient

Suppose

Feature	Value
Pregnancies	2
Glucose	135
BloodPressure	75
SkinThickness	28
Insulin	100
BMI	31.2
Pedigree	0.45
Age	40

```
new_patient = [[2,135,75,28,100,31.2,0.45,40]]

prediction = model.predict(new_patient)

print(prediction)
```

Output

[1]

Meaning

Patient is predicted to have **Diabetes**.

Step 14: Probability Prediction

probability = model.predict_proba(new_patient)

print(probability)

Example Output

[[0.18 0.82]]

Meaning

Class	Probability
No Diabetes	18%
Diabetes	82%

The model predicts **Diabetes** because it has the higher probability.

Step 15: Display Class Labels

print(model.classes_)

Output

[0 1]

Meaning

- 0 → No Diabetes
- 1 → Diabetes

Complete Program

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load Dataset
df = pd.read_csv("diabetes.csv")

# Features
X = df.drop("Outcome", axis=1)

# Target
y = df["Outcome"]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
  X,
  y,
  test_size=0.20,
  random_state=42
)

# Create Model
model = GaussianNB()

# Train Model
model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)

# Accuracy
print("Accuracy :", accuracy_score(y_test, prediction))

# Confusion Matrix
print(confusion_matrix(y_test, prediction))

# Classification Report
```

```
print(classification_report(y_test, prediction))
```

Expected Output

Accuracy : 0.88

Confusion Matrix

[[84 15]

[11 44]]

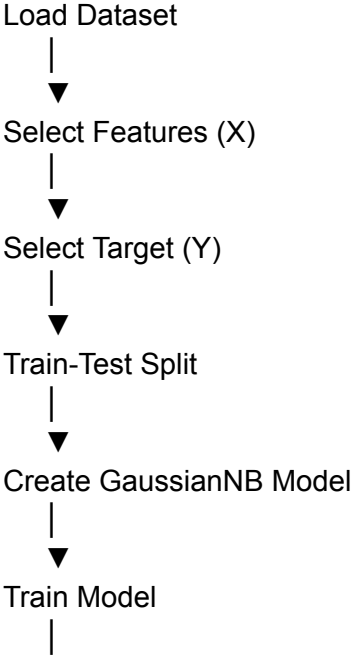
Precision : 0.85

Recall : 0.86

F1 Score : 0.85

Note: The actual output may vary depending on the dataset and train-test split.

Practical Workflow



▼
Prediction
|
▼
Evaluate Model
|
▼
Predict New Patient

GaussianNB vs MultinomialNB vs BernoulliNB

Algorithm	Best Used For	Example
GaussianNB	Continuous numerical data	Diabetes Prediction
MultinomialNB	Count/Frequency data	Spam Detection
BernoulliNB	Binary (0/1) features	Word Present/Absent

Applications

- Disease Prediction
- Email Spam Detection
- News Classification
- Sentiment Analysis
- Fake News Detection
- Document Classification
- Language Detection
- Product Recommendation

Observation

- **Gaussian Naïve Bayes** is suitable for datasets containing **continuous numerical features**.
- It is fast to train and computationally efficient.
- The algorithm predicts the class with the **highest posterior probability**.
- Model performance is evaluated using **Accuracy**, **Confusion Matrix**, **Precision**, **Recall**, and **F1-Score**.
- Naïve Bayes performs particularly well for **classification problems** and is widely used in **text classification** and **medical diagnosis**.
